

Graphs + Union-Find (DSU)

An interview study guide built from a Socratic-style walkthrough

This guide is built from a ChatGPT dialogue ("Teach Graph Fundamentals") that worked through graphs and Disjoint Set Union one concept at a time, Socratic-style: intuition first, then a reasoning question, then a derivation, then code. The chat's own opening framing is worth keeping because it is the right way to *use* this guide too:

I want to eventually look at a problem and think: "This is BFS because...", "This is DSU because...", "This is topological sort because..." -- rather than simply memorizing templates.

Graphs is the broadest topic in this series because it isn't really one technique -- it's a *family* of techniques (traversal, connectivity, ordering, shortest paths, minimum connection cost) that all share the same starting move: **model the problem as nodes + edges**, then ask which of those techniques the resulting structure actually needs.

Roadmap

1. **Fundamentals** -- what a graph is, why so many non-obvious problems are secretly graphs, and the three representations (adjacency list, adjacency matrix, edge list).
2. **DFS patterns** -- connected components, undirected cycle detection via parent-tracking.
3. **Bipartite graphs** -- 2-coloring and the odd-cycle connection.
4. **Topological sort** -- Kahn's algorithm, a DFS-postorder alternative, and directed cycle detection two different ways -- including a real correction moment from the source chat.
5. **DSU / Union-Find** -- parent arrays, path compression, union by size/rank, and when DSU beats DFS/BFS.
6. **The DSU problem set** -- connected components, Redundant Connection, Graph Valid Tree, Accounts Merge, Most Stones Removed, Number of Islands II, Satisfiability of Equality Equations.
7. **Shortest paths** -- BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, and precisely which assumption each one relies on.
8. **Minimum Spanning Tree** -- Kruskal vs Prim, and the virtual-source modeling trick.
9. **Hidden-state graphs** -- Word Ladder, bidirectional BFS, and recognizing a graph when the problem never uses the word "graph".

10. **Constrained shortest paths** -- Cheapest Flights Within K Stops, and the "expand the state" pattern in general.
11. **Eulerian paths** -- Reconstruct Itinerary and Hierholzer's algorithm.
12. **Alien Dictionary** -- inferring a graph from string ordering.
13. **Hard / mixed problems** -- Network Delay Time, minimax Dijkstra, Find Eventual Safe States, Course Schedule IV, and more.
14. **Recognition patterns + the final decision guide.**
15. **Interview Q&A drills.**

What this guide deliberately covers as extensions

The source chat's own opening plan (Phase 5 and Phase 8) promised **three-state DFS cycle detection** ("visited vs. currently exploring vs. completely processed") for directed graphs and a **DFS-based topological sort** ("explain why postorder matters") -- but the actual delivered conversation only ever used **Kahn's algorithm** for both directed-cycle detection and topological ordering. Both promised techniques are real, commonly expected in interviews, and genuinely useful to compare against Kahn's -- so Part 4 below builds both from scratch as a documented gap-fill, with `sol.py` functions (`topo_sort_dfs` , `has_cycle_directed_dfs`) tested against the Kahn's-algorithm versions on hundreds of random graphs.

What this guide deliberately skips

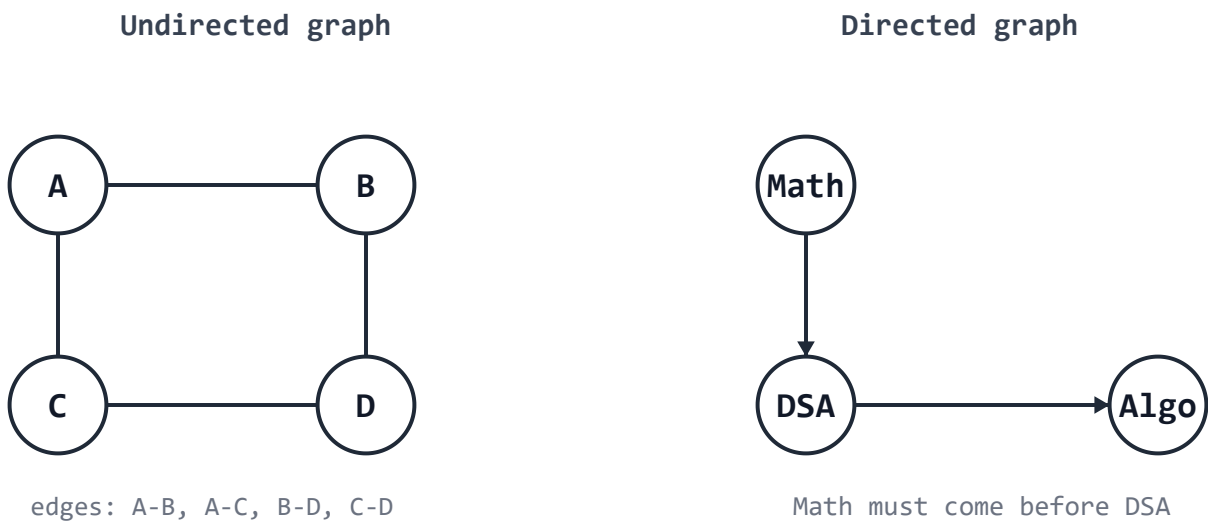
The source chat explicitly deferred **Bridges / Articulation Points** and **Strongly Connected Components (Tarjan/Kosaraju)** to a future session ("the remaining advanced topics -- bridges, articulation points, SCCs, etc. -- can wait as you said"). This guide does the same and does not cover them.

Part 1: Graph Fundamentals

1.1 What is a graph?

A graph has two pieces: **vertices** (the things) and **edges** (the relationships between things). That's the entire model:

Graph = Nodes + Relationships



The directed example above (Math → DSA) captures something important: the arrow means "must come before" -- direction encodes an asymmetric relationship, and reversing an arrow would change the meaning.

1.2 Recognizing a "secretly a graph" problem

The single highest-value skill in this whole topic is this: **many problems that never say the word "graph" are still graphs.**

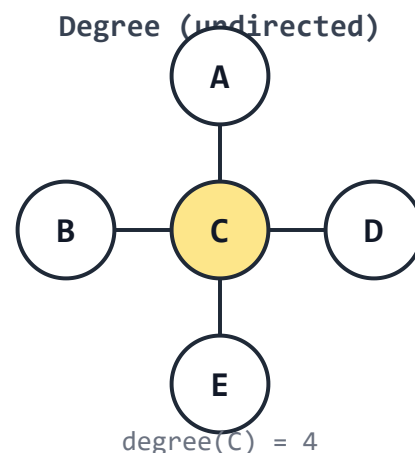
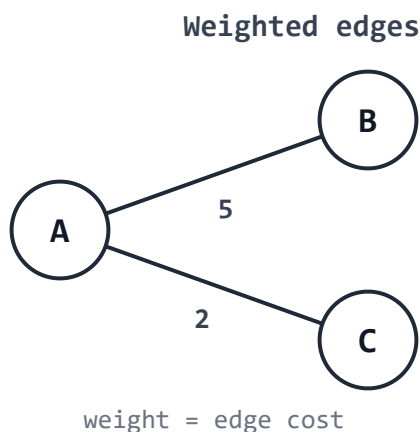
Problem	Graph interpretation
Cities + roads	cities = nodes, roads = edges
People + friendships	people = nodes, friendships = edges
Courses + prerequisites	courses = nodes, prerequisites = directed edges
Computers + connections	computers = nodes, connections = edges

Problem	Graph interpretation
Grid + movement	cells = nodes, valid moves = edges
Web pages + links	pages = nodes, links = directed edges

A cell in a grid is a node, and its (up to 4) neighboring cells are its edges -- this is why flood fill, mazes, and island-counting are graph problems even though the input is a 2D array, not an edge list. We'll build on this "hidden graph" idea heavily in Part 9.

1.3 Directed vs. undirected, weighted vs. unweighted

- **Undirected:** $A-B$ means the relationship works both ways (a friendship, a two-way road).
- **Directed:** $A \rightarrow B$ means the relationship only goes one way (a prerequisite, a follow relationship). $A \rightarrow B$ does **not** imply $B \rightarrow A$.
- **Unweighted:** only *whether* a connection exists matters.
- **Weighted:** each edge carries a cost (distance, time, money). This distinction becomes central later: **unweighted shortest path** $\rightarrow$ BFS, but weighted shortest path needs Dijkstra, 0-1 BFS, or Bellman-Ford depending on what the weights look like.



1.4 Degree, path, cycle, connected component

- **Degree** (undirected): how many edges touch a node. Directed graphs split this into **indegree** (edges coming in) and **outdegree** (edges going out) -- indegree becomes the foundation of topological sort in Part 4.
- **Path:** a sequence of vertices where every consecutive pair has an edge between them.
- **Cycle:** a path that returns to where it started.
- **Connected component:** a maximal group of nodes where every node can reach every other node. Critically, "can reach" doesn't require a *direct* edge -- $A-B-C$ puts A and C in the same

component even though there's no A-C edge. This is the seed of the very first pattern:
DFS/BFS → find components, DSU → maintain components.

1.5 Representation: adjacency list, matrix, edge list

Three ways to store a graph, each answering a different question best:

```
def build_adjacency_list(n, edges, directed=False):
    """edges = [(u, v), ...] → adjacency list over 0..n-1."""
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        if not directed:
            graph[v].append(u)
    return graph
```

```
def build_weighted_adjacency_list(n, edges, directed=False):
    """edges = [(u, v, w), ...] → weighted adjacency list."""
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
        if not directed:
            graph[v].append((u, w))
    return graph
```

```
def adjacency_matrix(n, edges, directed=False):
    mat = [[0] * n for _ in range(n)]
    for u, v in edges:
        mat[u][v] = 1
        if not directed:
            mat[v][u] = 1
    return mat
```

Representation	Answers	Space	Best for
Adjacency list	"What are this node's neighbors?"	$O(V+E)$	the usual default -- traversal
Adjacency matrix	"Is there a direct edge between u and v?"	$O(V^2)$	dense/small graphs, $O(1)$ edge lookup
Edge list	"Give me all the edges"	$O(E)$	sorting/processing edges -- Kruskal

The reasoning behind adjacency list being the default: for V nodes with only a few neighbors each, a matrix wastes space storing V^2 entries even though almost all of them are 0 (no edge). An adjacency list only stores the edges that actually exist.

One detail that trips people up: for an **undirected weighted** edge $A -5- B$, the adjacency list stores the pair in *both* directions -- $A: [(B, 5)]$ and $B: [(A, 5)]$ -- because you can travel either way and either traversal needs to see the weight. A **directed** edge only gets stored once, from source to destination.

Part 2: DFS Patterns -- Components and Cycles

2.1 One DFS call consumes one whole component

The key reasoning question: how can a single DFS call tell us anything about an entire connected component?

Every time I start a new DFS from an unvisited node, I have discovered a new connected component.

DFS(A) consumes one whole component before moving on



`visited = {A,B,C}` after `DFS(A)`; a fresh `DFS(D)` starts the 2nd component
components = number of times we start a NEW DFS from an unvisited node

Walking the outer loop:

```
def count_components(n, edges):
    """Number of connected components via DFS (undirected graph)."""
    graph = build_adjacency_list(n, edges)
    visited = [False] * n
    count = 0

    def dfs(node):
        visited[node] = True
        for nei in graph[node]:
            if not visited[nei]:
                dfs(nei)

    for node in range(n):
        if not visited[node]:
            count += 1
            dfs(node)

    return count
```

The insight isn't the code -- it's the invariant: **the number of times the outer loop starts a fresh DFS equals the number of connected components**. Once `DFS(A)` finishes, every node it reached is marked visited, so the loop skips straight past them and only starts a new DFS when it hits a node no previous call has touched.

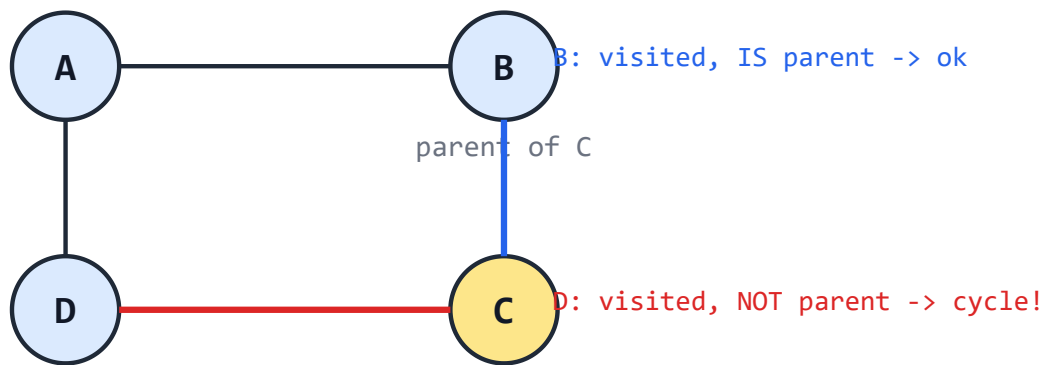
Recognition: "How many groups/components/clusters are there?" -> translate to connected components -> DFS/BFS (or DSU, covered in Part 5).

2.2 Undirected cycle detection: why "visited" alone is wrong

A natural first instinct is "if DFS sees an already-visited node, there's a cycle." That's **wrong** for undirected graphs. Consider just `A-B`: starting DFS at `A`, we move to `B`; `B` looks at its neighbors and sees `A` -- already visited! But there's obviously no cycle in a single edge.

The missing piece: `B` needs to know that `A` is specifically the node it *came from* (its parent), not some other already-explored node reached through a different route.

DFS: A -> B -> C, then C looks at its neighbors



a visited neighbor is only a cycle if it ISN'T the node we came from

```
def has_cycle_undirected(n, edges):  
    """Cycle detection in an undirected graph: DFS + parent tracking.  
  
    The key invariant: a visited neighbor that is NOT our parent means  
    we've found another route back to an already-explored node -> cycle.  
    """  
    graph = build_adjacency_list(n, edges)  
    visited = [False] * n  
  
    def dfs(node, parent):  
        visited[node] = True  
        for nei in graph[node]:  
            if not visited[nei]:  
                if dfs(nei, node):  
                    return True  
            elif nei != parent:  
                return True  
        return False  
  
    for node in range(n):  
        if not visited[node]:  
            if dfs(node, -1):  
                return True  
    return False
```

The invariant, stated precisely:

For undirected DFS: if we encounter a visited neighbor that is **not** our parent, we've found a cycle.

Why does a non-parent visited neighbor mean a cycle specifically? Because if **D** is already visited but isn't **C** 's parent, there must already be *another* route connecting **C** 's current DFS path back to **D** -- and a second route between two nodes that already have one route between them is exactly what a cycle is.

Full trace:

at node	neighbor	neighbor state	action
A	B	unvisited	DFS into it
B	A	visited, IS parent	ignore
B	C	unvisited	DFS into it
C	B	visited, IS parent	ignore
C	D	unvisited	DFS into it
D	A	visited, NOT parent	CYCLE FOUND
D	C	visited, IS parent	ignore
A	D	visited, NOT parent	CYCLE FOUND

One important scoping note: this parent-based rule is specific to **undirected** graphs. A directed graph needs a different approach (three states, not just visited/parent) because **A → B → C** and **C → A** forming a cycle doesn't have the same "came from" symmetry that an undirected edge does -- we cover this properly once topological sort gives us the vocabulary for it, in Part 4.

Part 3: Bipartite Graphs -- 2-Coloring

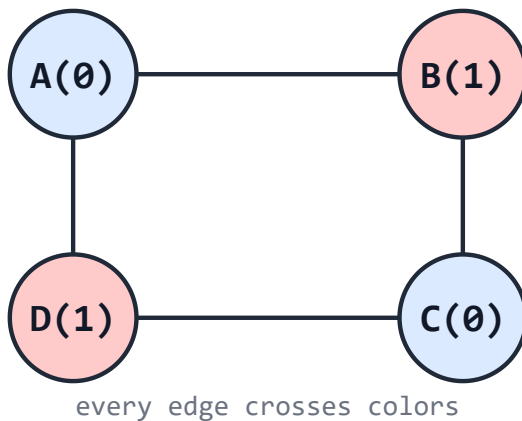
3.1 The invariant

A graph is bipartite if we can split its nodes into two groups such that **every edge connects a node from group 1 to a node from group 2** -- never two nodes from the same group. Concretely: assign color 0 to a starting node, force every neighbor to color 1 , force every one of *their* neighbors back to 0 , and so on.

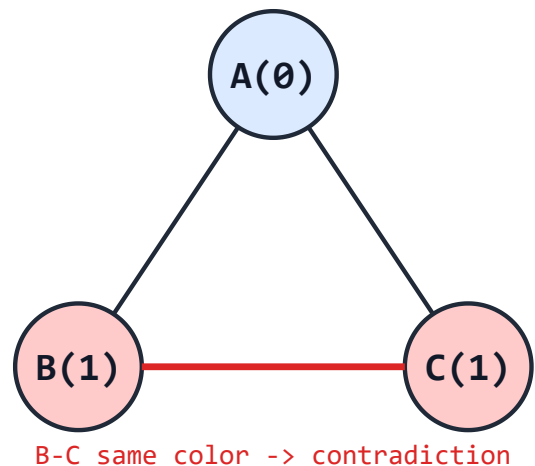
Every edge must connect nodes of opposite colors.

Why is this useful? A common wording is "can we split these people into two teams such that no two enemies share a team?" -- model people as nodes, "enemy" relationships as edges, teams as two colors, and the question becomes exactly "can we 2-color this graph?"

Square: 2-colorable



Triangle: not colorable



3.2 Why an odd cycle breaks it

Walk a 5-cycle $A-B-C-D-E-A$, alternating colors as you go: $A=0$, $B=1$, $C=0$, $D=1$, $E=0$. The closing edge is $E-A$, but both ended up 0 -- contradiction. The cycle has 5 edges (odd), and alternating colors around an odd loop can never return to a different color than where you started. A 4-cycle (even) closes consistently.

A graph is bipartite exactly when it contains no odd-length cycle.

3.3 Code -- BFS and DFS versions

```
def is_bipartite(graph):
    """2-color the graph via BFS. graph[i] = list of neighbors of i."""
    n = len(graph)
    color = [-1] * n

    for start in range(n):
        if color[start] != -1:
            continue
        color[start] = 0
        queue = deque([start])
        while queue:
            node = queue.popleft()
            for nei in graph[node]:
                if color[nei] == -1:
                    color[nei] = 1 - color[node]
                    queue.append(nei)
                elif color[nei] == color[node]:
                    return False
        return True
```

The DFS version enforces the exact same invariant, just via recursion instead of a queue:

```
def is_bipartite_dfs(graph):
    """Same check, DFS coloring instead of BFS (equally valid)."""
    n = len(graph)
    color = [-1] * n

    def dfs(node, c):
        color[node] = c
        for nei in graph[node]:
            if color[nei] == -1:
                if not dfs(nei, 1 - c):
                    return False
            elif color[nei] == color[node]:
                return False
        return True

    for start in range(n):
        if color[start] == -1:
            if not dfs(start, 0):
                return False
    return True
```

Recognition: "two groups", "two teams", "no two X share a group", "can this be properly 2-colored" -> bipartite check. Complexity is $O(V+E)$ either way -- one pass over every node and edge.

Part 4: Topological Sort and Directed Cycles

4.1 The core question: dependency ordering

Topological sort answers a different kind of question than anything so far: not "who can reach whom?" but "**what order can these things be done in?**" -- course prerequisites, build systems, task dependencies.

4.2 Kahn's algorithm, derived from indegree

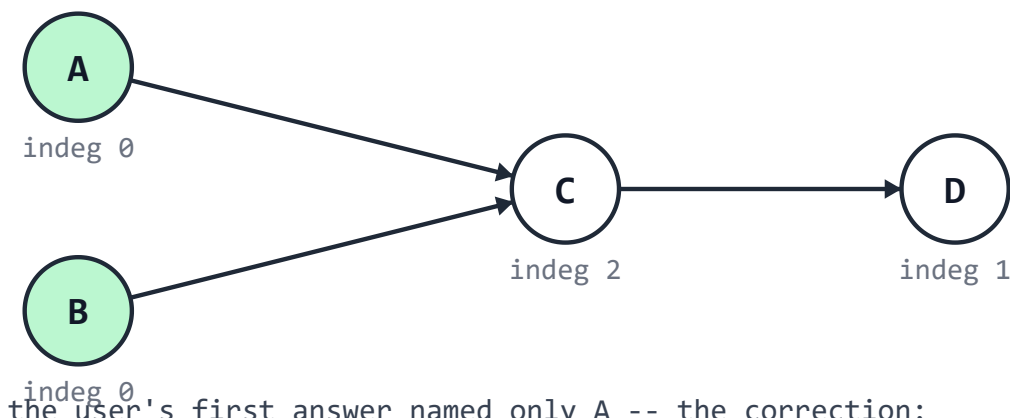
The foundation: a node with **indegree 0** has nothing blocking it, so it's always safe to process first.

A correction worked through live in the source chat

Given the dependency graph $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$ ("interpret each arrow as 'must be completed before'"), the question was: which nodes are immediately safe to do first?

The first answer was "**A**" -- correct, but incomplete. The actual indegrees are $A: 0$, $B: 0$, $C: 2$, $D: 1$. **Both A and B** have indegree 0, so *both* are valid to process first, not just A.

$A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$



BOTH A and B have indegree 0, so both are valid to process first

This matters beyond just this one example: **Kahn's algorithm can start its queue with more than one node**, and any order that respects the dependency edges is a valid answer -- A, B, C, D

and **B, A, C, D** are both correct topological orders of this graph. A problem that expects a *unique* answer either guarantees the input has one, or (more often) will accept *any* valid ordering.

Full trace, confirming both A and B start in the queue:

event	affects	new indegree	note
initial	-	-	indegrees: {'A': 0, 'B': 0, 'C': 2, 'D': 1}
initial queue	-	-	['A', 'B'] (BOTH A and B, not just A)
process A	C	1	still waiting
process B	C	0	enqueue
process C	D	0	enqueue

4.3 Kahn's algorithm -- code

```
def topo_sort_kahn(n, edges):
    """Kahn's algorithm. edges = [(u, v)] meaning u must come before v.
    Returns [] if a cycle exists (no valid ordering)."""
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    queue = deque(node for node in range(n) if indegree[node] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                queue.append(nei)

    return order if len(order) == n else []
```

The invariant: **the queue always contains exactly the nodes whose prerequisites have all been satisfied**. Removing a zero-indegree node and decrementing its neighbors' indegree is the whole algorithm.

4.4 Directed cycle detection, two ways

If Kahn's algorithm processes fewer than **n** nodes, some nodes never reached indegree 0 -- meaning there's a cycle blocking them forever:

```
def has_cycle_directed_kahn(n, edges):  
    """A directed graph has a cycle iff Kahn's algorithm cannot process  
    every node (some nodes never reach indegree 0)."""  
    return len(topo_sort_kahn(n, edges))  $\neq$  n
```

An extension the source chat's own plan promised but never delivered

The chat's opening outline (Phase 5) explicitly promised **"recursion stack / three-state DFS"** for directed cycle detection, distinguishing `visited` / `currently exploring` / `completely processed` -- but the actual conversation only ever used the Kahn's-algorithm check above. The three-color DFS approach is standard, commonly expected as an alternative in interviews, and worth knowing on its own:



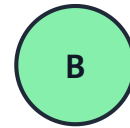
WHITE

never visited



GRAY

on the current recursion stack



BLACK

fully processed, safe

seeing a GRAY neighbor = we looped back onto our own path -> cycle

seeing a BLACK neighbor is fine -- that subtree was already proven acyclic

```

def has_cycle_directed_dfs(n, edges):
    """Three-color / recursion-stack DFS cycle detection.

    EXTENSION: promised in the source chat's plan ('recursion stack /
    three-state DFS ... visited vs currently exploring vs completely
    processed') but never actually taught -- the delivered conversation
    only used the Kahn's-algorithm-based check above. This is the classic
    alternative and the one most interviewers expect you to also know:

        WHITE (0) = never visited
        GRAY  (1) = currently on the recursion stack (being explored)
        BLACK (2) = fully processed, cannot lead back to anything on the
                    current stack

    Seeing a GRAY neighbor means we've looped back onto our own current
    path → cycle. Seeing a BLACK neighbor is fine: that subtree was
    already proven acyclic.
    """
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)

    WHITE, GRAY, BLACK = 0, 1, 2
    state = [WHITE] * n

    def dfs(node):
        state[node] = GRAY
        for nei in graph[node]:
            if state[nei] == GRAY:
                return True
            if state[nei] == WHITE and dfs(nei):
                return True
        state[node] = BLACK
        return False

    for node in range(n):
        if state[node] == WHITE:
            if dfs(node):
                return True
    return False

```

Seeing a **GRAY** neighbor means the current DFS path has looped back onto itself -- a cycle. Seeing a **BLACK** neighbor is always safe: that entire subtree was already proven acyclic by a previous call, so it can never lead back to anything currently on the stack.

4.5 DFS-based topological sort (the other promised extension)

The chat's plan (Phase 8) also promised a DFS-based topological sort ("explain why postorder matters") that never got taught either. The idea: a node is only appended to the result once **every one of its descendants has already been fully explored** (postorder). Reversing that list gives a

valid dependency order, for the same underlying reason Hierholzer's algorithm (Part 11) builds its result backwards.

```
def topo_sort_dfs(n, edges):
    """DFS-based topological sort via postorder + reversal.

    EXTENSION: the source chat's own plan (Phase 8) explicitly promised
    'DFS Topological Sort -- explain why postorder matters', but the
    delivered conversation only ever covered Kahn's algorithm. This fills
    that gap: a node is only appended to the order once every one of its
    descendants has already been fully explored (post-order), so reversing
    that list gives a valid dependency order for the same reason
    Reconstruct Itinerary's Hierholzer trick works.

    Returns [] if a cycle exists.
    """
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)

    WHITE, GRAY, BLACK = 0, 1, 2
    state = [WHITE] * n
    order = []
    has_cycle = [False]

    def dfs(node):
        state[node] = GRAY
        for nei in graph[node]:
            if state[nei] == WHITE:
                dfs(nei)
                if has_cycle[0]:
                    return
            elif state[nei] == GRAY:
                has_cycle[0] = True
                return
        state[node] = BLACK
        order.append(node)

    for node in range(n):
        if state[node] == WHITE:
            dfs(node)
            if has_cycle[0]:
                return []

    order.reverse()
    return order
```

Both `topo_sort_kahn` and `topo_sort_dfs` were tested against each other on hundreds of random DAGs (and random graphs with an injected cycle) in `test_sol.py` -- they always agree on whether a cycle exists, and each produces an ordering that independently satisfies every edge constraint.

4.6 Course Schedule I & II

The textbook application of Kahn's algorithm:

```
def can_finish(num_courses, prerequisites):
    """LC 207 Course Schedule -- Kahn's algorithm."""
    graph = [[] for _ in range(num_courses)]
    indegree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    queue = deque(i for i in range(num_courses) if indegree[i] == 0)
    completed = 0
    while queue:
        node = queue.popleft()
        completed += 1
        for nei in graph[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                queue.append(nei)

    return completed == num_courses
```

```
def find_order(num_courses, prerequisites):
    """LC 210 Course Schedule II -- Kahn's, return the order itself."""
    graph = [[] for _ in range(num_courses)]
    indegree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    queue = deque(i for i in range(num_courses) if indegree[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                queue.append(nei)

    return order if len(order) == num_courses else []
```

Recognition: "prerequisite", "dependency", "before", "ordering", "scheduling" -> directed graph -> topological sort / cycle detection. Complexity for all of the above: $O(V+E)$ time, $O(V+E)$ space.

Part 5: DSU / Union-Find -- From the Problem First

5.1 The problem DSU solves

DFS/BFS explores a graph that's already fully given. DSU answers a different question: **as relationships arrive one at a time, can we quickly tell whether two nodes end up in the same group?**

Imagine 100,000 nodes, 1,000,000 connections arriving one by one, and thousands of "are X and Y connected?" queries interleaved with them. Re-running DFS/BFS after every new connection would be wasteful -- DSU is built specifically for this repeated *connect* + *query* pattern.

DFS/BFS thinks "let me explore the graph." DSU thinks "let me maintain the groups."

5.2 Parent array and `find`

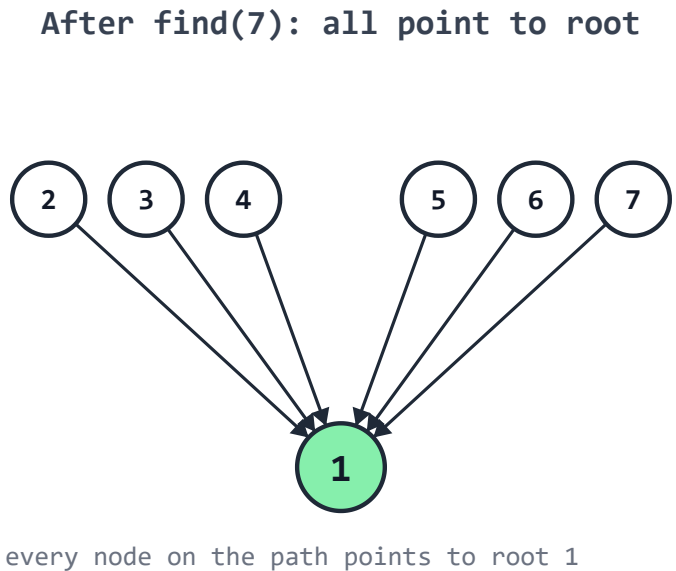
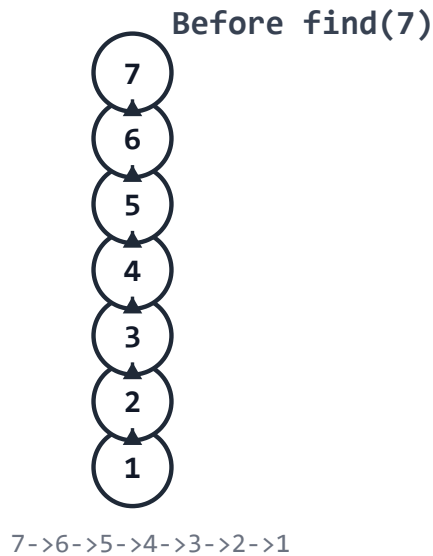
Every node starts as its own group representative: `parent[i] = i`. Connecting `1` and `2` means making one of them point at the other -- `parent[2] = 1` -- and connecting `3` to that group means `parent[3] = 1` (or chaining through `2`). The **root** of a chain is the group's representative, and `find(x)` walks up parent pointers until it hits a node that is its own parent.

`find(x)` returns the representative (root) of the set containing `x`.

Two nodes are in the same group exactly when `find(a) == find(b)` -- never compare `parent[a] == parent[b]` directly, since parent only points one step up, not all the way to the root.

5.3 Path compression

A naive chain like `7 → 6 → 5 → 4 → 3 → 2 → 1` makes `find(7)` walk 6 hops. If we repeatedly call `find` on nodes deep in a tall chain, we keep re-walking the same path.



The fix: once `find(7)` discovers the root is `1`, make every node on that path point **directly** at the root.

```
def find(x):  
    if parent[x] != x:  
        parent[x] = find(parent[x])  
    return parent[x]
```

`parent[x] = find(parent[x])` does two things in one line: it finds the root recursively, *and* it rewrites `x`'s parent to be that root on the way back out of the recursion -- so every future `find` on any node along that path is now a single hop.

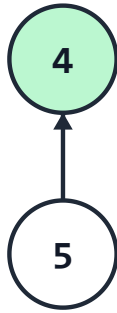
5.4 Union by size (and union by rank)

Merging two groups means connecting their **roots**, not arbitrary member nodes --

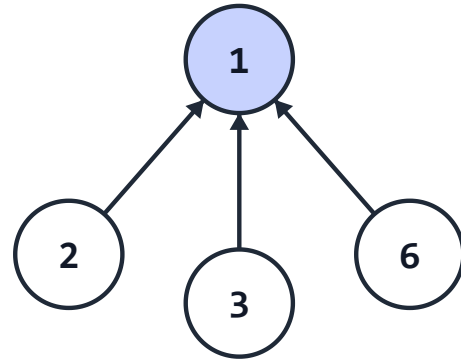
`parent[root_a] = root_b` correctly merges two entire sets, but `parent[a] = b` for two non-root nodes would not.

But repeated unions can still build a tall chain if we're not careful about *which* root becomes the parent of which:

Smaller tree (size 2)



Larger tree (size 5)



attach the SMALLER tree's root under the LARGER tree's root:
parent[4] = 1 (not the other way around -- keeps trees flat)

Attach the smaller tree under the larger tree.

```
if size[root_a] < size[root_b]:  
    parent[root_a] = root_b  
    size[root_b] += size[root_a]  
else:  
    parent[root_b] = root_a  
    size[root_a] += size[root_b]
```

Union by rank is the same idea tracking an approximate tree *height* instead of node *count*: attach the lower-rank root under the higher-rank root, and only increment rank when merging two equal-rank trees (since that's the one case that actually grows the height). Either convention works equally well in interviews; union by size is usually easier to reason about because the intuition ("fewer nodes to drag along") is more direct.

5.5 The full DSU class

```

class DSU:
    """Union by size + path compression. Amortized  $\sim O(\alpha(n))$  per op."""

    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.count = n # number of disjoint sets remaining

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        root_a = self.find(a)
        root_b = self.find(b)
        if root_a == root_b:
            return False

        if self.size[root_a] < self.size[root_b]:
            root_a, root_b = root_b, root_a

        self.parent[root_b] = root_a
        self.size[root_a] += self.size[root_b]
        self.count -= 1
        return True

    def connected(self, a, b):
        return self.find(a) == self.find(b)

```

Full trace (`connect(1,2)` , `connect(2,3)` , `connect(4,5)` , `connect(3,4)` , then `find(1)` vs `find(5)`):

operation	result	parent[] snapshot (1..5)
union(1,2)	True	{1: 1, 2: 1, 3: 3, 4: 4, 5: 5}
union(2,3)	True	{1: 1, 2: 1, 3: 1, 4: 4, 5: 5}
union(4,5)	True	{1: 1, 2: 1, 3: 1, 4: 4, 5: 4}
union(3,4)	True	{1: 1, 2: 1, 3: 1, 4: 1, 5: 4}
find(1)	1	{1: 1, 2: 1, 3: 1, 4: 1, 5: 1}
find(5)	1	{1: 1, 2: 1, 3: 1, 4: 1, 5: 1}
connected(1,5)?	True	-

5.6 Complexity

With **both** path compression and union by size/rank, each operation is $O(\alpha(n))$ amortized, where α is the inverse Ackermann function -- for any practical input size, $\alpha(n)$ is at most 4 or 5, so this is described as "effectively constant." For E union/find calls total: $O(E * \alpha(V))$, in practice just $O(E)$. Space: $O(V)$.

Note the source chat never claimed a complexity bound *without* both optimizations -- but it's worth stating explicitly since it's a common interview follow-up: with path compression alone (no union by size/rank), worst-case `find` is $O(\log n)$ amortized; with union by size/rank alone (no path compression), it's also $O(\log n)$. Only the **combination** gets you down to the near-constant $O(\alpha(n))$ bound.

5.7 DSU vs. DFS/BFS -- when each is natural

Situation	Use
Graph already fully given, need to explore/traverse it	DFS/BFS
Edges arrive one at a time, repeated "same group?" queries	DSU
"Will adding this edge create a cycle?"	DSU (<code>find(u)==find(v)</code>)
Need the actual shortest path / distance	neither -- see Part 7
Need traversal order, levels, or full component membership	DFS/BFS
Minimum spanning tree	Kruskal uses DSU, Prim uses a heap

The one-line mental model from the source chat:

DFS/BFS explores an existing graph; DSU maintains connected components as relationships change.

A critical limitation to remember: **DSU only knows connectivity, not distance or path.** After `union(A, B)`, DSU can answer "are A and B connected?" but never "what is the shortest path from A to B?" -- that question needs Part 7's machinery, not DSU.

Part 6: The DSU Problem Set

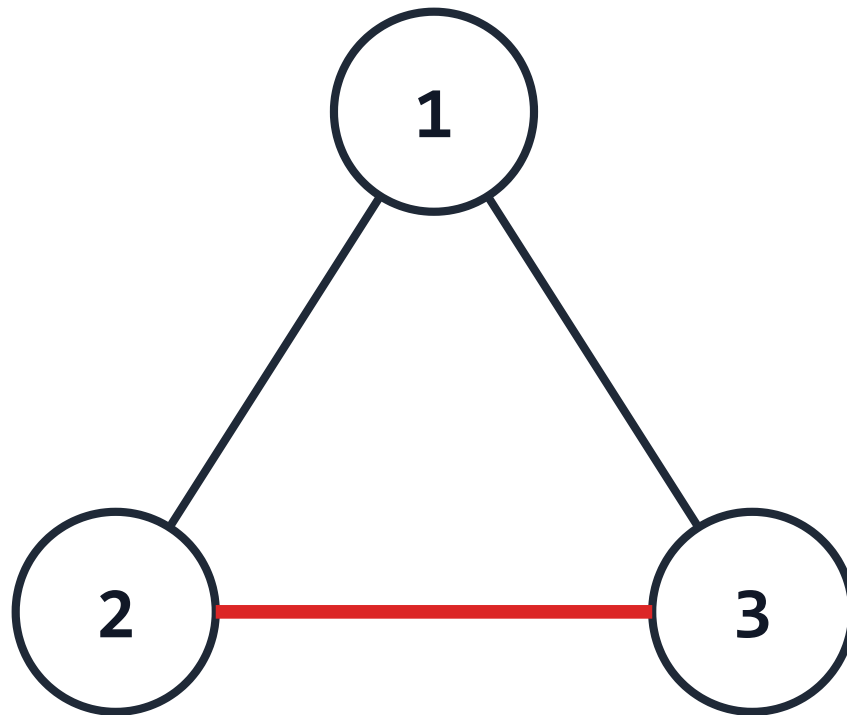
Every problem here reduces to one of these five recognition triggers: "are these already connected?" (`find`), "merge these groups" (`union`), "does this edge create a cycle?" (`find(a)==find(b)` before union), "how many groups remain?" (successful unions / roots), or "things sharing X belong together" (model X itself as a node).

6.1 Connected Components, Redundant Connection, Graph Valid Tree

```
def count_components_dsu(n, edges):  
    """Same answer as count_components(), via DSU instead of DFS."""  
    dsu = DSU(n)  
    components = n  
    for a, b in edges:  
        if dsu.union(a, b):  
            components -= 1  
    return components
```

For **Redundant Connection** (LC 684): process edges one at a time; the first edge whose `union` call *fails* (both endpoints already connected) is the redundant one -- it's the edge that would have closed a cycle.

edges: [1,2] [1,3] [2,3]



[2,3]: find(2)==find(3) already

```
def find_redundant_connection(edges):  
    """LC 684 -- the edge that, when unioned, fails (already connected)."""  
    n = len(edges)  
    dsu = DSU(n + 1)  
    for a, b in edges:  
        if not dsu.union(a, b):  
            return [a, b]  
    return []
```

Graph Valid Tree (LC 261) combines two facts: a tree on `n` nodes has exactly `n-1` edges, and has no cycle. Checking the edge count first is a cheap short-circuit; then union every edge and fail if any union fails.

```
def valid_tree(n, edges):
    """LC 261 Graph Valid Tree -- exactly n-1 edges AND no cycle."""
    if len(edges) != n - 1:
        return False
    dsu = DSU(n)
    for a, b in edges:
        if not dsu.union(a, b):
            return False
    return True
```

Number of Provinces (LC 547) is the same pattern applied to an adjacency matrix instead of an edge list:

```
def find_circle_num(is_connected):
    """LC 547 Number of Provinces -- DSU over an adjacency matrix."""
    n = len(is_connected)
    dsu = DSU(n)
    components = n
    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j]:
                if dsu.union(i, j):
                    components -= 1
    return components
```

6.2 Accounts Merge -- "shared identifier -> same component"

The recognition move here is the important one: this doesn't look like a graph problem on the surface, but the core idea generalizes far beyond accounts: **two entities belong together if they share a common identifier**. Here the identifiers are emails.

```

def accounts_merge(accounts):
    """LC 721 -- DSU where nodes are emails, edges are 'same account'."""
    email_to_id = {}
    email_to_name = {}
    next_id = 0

    for name, *emails in accounts:
        for email in emails:
            if email not in email_to_id:
                email_to_id[email] = next_id
                email_to_name[email] = name
                next_id += 1

    dsu = DSU(next_id)
    for name, *emails in accounts:
        first = email_to_id[emails[0]]
        for email in emails[1:]:
            dsu.union(first, email_to_id[email])

    groups = defaultdict(list)
    for email, idx in email_to_id.items():
        groups[dsu.find(idx)].append(email)

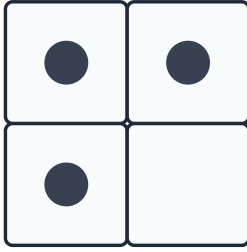
    result = []
    for emails in groups.values():
        emails.sort()
        result.append([email_to_name[emails[0]]] + emails)
    return result

```

6.3 Most Stones Removed -- model rows and columns, not stones

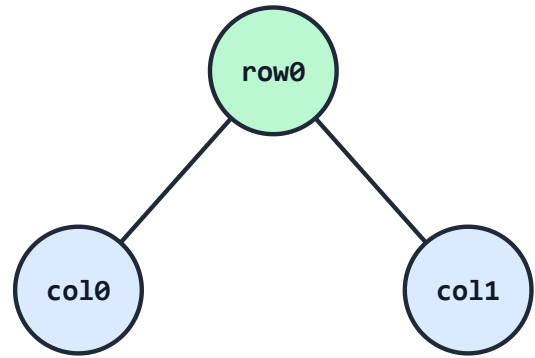
Another "the nodes aren't the objects in the problem" trick. Two stones are connected if they share a row *or* column, so instead of connecting stones to each other directly, connect each stone's **row** and **column** as two DSU nodes (offsetting column ids so they don't collide with row ids).

Stones on a grid



3 stones share row 0 or col 0

DSU model: rows & cols



component size 3 -> remove 2

```
def remove_stones(stones):
    """LC 947 -- model rows and columns (not stones) as DSU nodes."""
    rows = {}
    cols = {}
    next_id = 0
    dsu = DSU(2 * len(stones) + 5)

    for r, c in stones:
        if r not in rows:
            rows[r] = next_id
            next_id += 1
        if c not in cols:
            cols[c] = next_id
            next_id += 1
        dsu.union(rows[r], cols[c])

    roots = {dsu.find(rows[r]) for r, c in stones}
    return len(stones) - len(roots)
```

Within one connected component of k stones, exactly $k-1$ can be removed (you must always leave at least one stone standing per component), so the answer is $\text{total stones} - \text{number of components}$.

6.4 Number of Islands II -- dynamic connectivity

Land arrives one cell at a time, and after every addition we need the current island count. This is the textbook case for preferring DSU over re-running DFS/BFS from scratch on every step: every new land cell starts as its own island ($+1$), then a successful **union** with an already-land neighbor means two islands just became one (-1).

```

def num_islands_2(m, n, positions):
    """LC 305 -- dynamic connectivity: land arrives one cell at a time."""
    dsu = DSU(m * n)
    land = set()
    islands = 0
    result = []
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for r, c in positions:
        if (r, c) in land:
            result.append(islands)
            continue
        land.add((r, c))
        islands += 1
        current = r * n + c

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < m and 0 ≤ nc < n and (nr, nc) in land:
                neighbor = nr * n + nc
                if dsu.union(current, neighbor):
                    islands -= 1
        result.append(islands)

    return result

```

6.5 Satisfiability of Equality Equations -- order matters

Given a mix of `a==b` and `a≠c` constraints, the trick is processing order: **union every `==` constraint first**, then check every `≠` constraint for a contradiction (same root despite claiming inequality). Checking inequalities before all equalities are processed would miss constraints that only become contradictory once the full equality closure is known.

```

def equations_possible(equations):
    """LC 990 -- union all '==' first, then check every '≠' for a
    contradiction (same root → impossible)."""
    dsu = DSU(26)
    for eq in equations:
        if eq[1] == '=':
            dsu.union(ord(eq[0]) - ord('a'), ord(eq[3]) - ord('a'))
    for eq in equations:
        if eq[1] == '!=':
            if dsu.connected(ord(eq[0]) - ord('a'), ord(eq[3]) - ord('a')):
                return False
    return True

```

"must be together" -> UNION. "must be separate" -> CHECK (after all unions are done).

6.6 Number of Operations to Make Network Connected

A quick extension of the same "count components" idea, with one extra precondition: you need at least $n-1$ cables to connect n computers at all, so if fewer than $n-1$ connections exist, it's impossible regardless of how they're arranged. Otherwise, the answer is simply $\text{components} - 1$ - each extra operation can merge exactly two components using a genuinely spare cable.

```
def make_connected(n, connections):
    """LC 1319 -- need  $\geq n-1$  cables to connect everything; the number of
    extra operations needed equals  $(\text{components} - 1)$ ."""
    if len(connections) < n - 1:
        return -1
    dsu = DSU(n)
    for u, v in connections:
        dsu.union(u, v)
    return dsu.count - 1
```

Part 7: Shortest Paths

7.1 The governing question

Different graph properties require different shortest-path algorithms.

The whole family sits on one decision ladder, and the reasoning behind each rung matters more than memorizing which name goes where:

GRAPH problem

need reachability / groups?

explore nodes

-> DFS / BFS

repeated merge/query

-> DSU

need shortest path?

unweighted

-> BFS

0/1 weights

-> 0-1 BFS

non-negative

-> Dijkstra

negative edges

-> Bellman-Ford

need ALL-pairs shortest paths?

-> Floyd-Warshall

(see the MST / dependency / hidden-graph tree for the rest)

7.2 BFS -- why it gives shortest paths in an unweighted graph

BFS explores strictly by increasing distance: everything at distance 0, then everything at distance 1, then distance 2, and so on. Because of that, **the first time BFS reaches a node is necessarily via a shortest path** -- any later discovery of the same node would have to come from an equal or greater distance, so overwriting is never needed.

```
def shortest_path_bfs(graph, start):
    """Unweighted shortest distances from start. -1 = unreachable."""
    n = len(graph)
    dist = [-1] * n
    dist[start] = 0
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for nei in graph[node]:
            if dist[nei] == -1:
                dist[nei] = dist[node] + 1
                queue.append(nei)
    return dist
```

Path reconstruction just adds a `parent` pointer set the first time each node is reached, then walks backward from the target and reverses:

```
def shortest_path_reconstruct(graph, start, target):
    """BFS + parent pointers → the actual shortest path as a node list."""
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == target:
            break
        for nei in graph[node]:
            if nei not in parent:
                parent[nei] = node
                queue.append(nei)

    if target not in parent:
        return []

    path = []
    cur = target
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    return path[::-1]
```

Recognition: "minimum number of moves/edges/steps", "shortest path in an unweighted graph" - > BFS.

7.3 Dijkstra -- why non-negative weights matter

The moment edges have different weights, "fewest edges" stops meaning "cheapest" -- a 2-edge path costing 20 can beat a 1-edge path costing 30. Dijkstra's core move: **always expand the node with the smallest currently-known distance**, and once you've selected it, treat that distance as final.

```
def dijkstra(graph, start, n=None):
    """graph[u] = [(v, weight), ...], non-negative weights."""
    if n is None:
        n = len(graph)
    dist = [float("inf")] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        current_dist, node = heapq.heappop(heap)
        if current_dist > dist[node]:
            continue
        for nei, weight in graph[node]:
            new_dist = current_dist + weight
            if new_dist < dist[nei]:
                dist[nei] = new_dist
                heapq.heappush(heap, (new_dist, nei))

    return dist
```

That finalization step is exactly where non-negative weights come in -- it's only safe to declare a popped node's distance final because no *future* edge relaxation could ever make it cheaper (every edge can only add a non-negative amount). The `if current_dist > dist[node]: continue` line discards **stale heap entries**: if a cheaper route to a node was found after an older, worse `(dist, node)` pair was already pushed, the old entry eventually gets popped too, but by then `dist[node]` has already improved, so it's safely skipped.

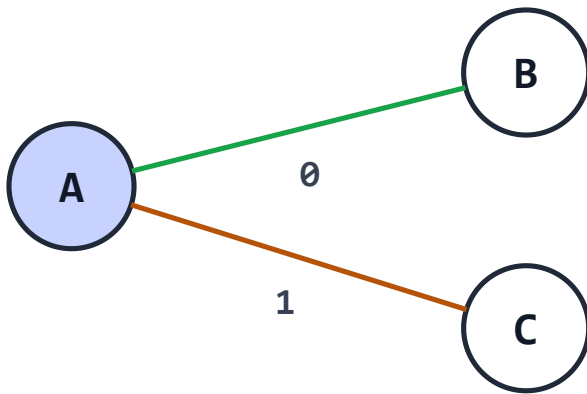
Full trace:

popped	dist popped with	action	dist[] after
A	0	pop & relax neighbors	{'A': 0, 'B': inf, 'C': inf, 'D': inf}
C	1	pop & relax neighbors	{'A': 0, 'B': 4, 'C': 1, 'D': inf}
B	3	pop & relax neighbors	{'A': 0, 'B': 3, 'C': 1, 'D': 9}
B	4	stale heap entry, skip	{'A': 0, 'B': 3, 'C': 1, 'D': 4}
D	4	pop & relax neighbors	{'A': 0, 'B': 3, 'C': 1, 'D': 4}
D	9	stale heap entry, skip	{'A': 0, 'B': 3, 'C': 1, 'D': 4}

7.4 0-1 BFS -- exploiting weights restricted to {0, 1}

When every edge costs exactly 0 or 1, a plain deque replaces the heap: push a 0-cost neighbor to the **front** (process it immediately, at the same distance frontier) and a 1-cost neighbor to the **back**.

0-1 BFS: 0-edge -> push FRONT, 1-edge -> push BACK



deque: front -> back



B (d=0) before C (d=1)

```
def zero_one_bfs(graph, start, n=None):
    """graph[u] = [(v, weight)], weight in {0, 1}. Deque instead of heap."""
    if n is None:
        n = len(graph)
    dist = [float("inf")] * n
    dist[start] = 0
    dq = deque([start])

    while dq:
        node = dq.popleft()
        for nei, weight in graph[node]:
            new_dist = dist[node] + weight
            if new_dist < dist[nei]:
                dist[nei] = new_dist
                if weight == 0:
                    dq.appendleft(nei)
                else:
                    dq.append(nei)

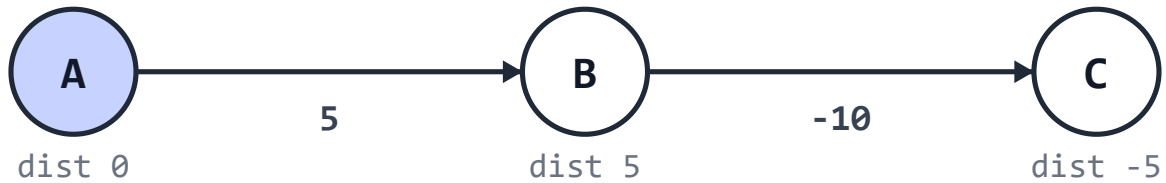
    return dist
```

Full trace:

event	deque after	dist[] after
start	[0]	{'A': 0, 'B': inf, 'C': inf}
relax A->B (w=0)	['B']	{'A': 0, 'B': 0, 'C': inf}
relax A->C (w=1)	['B', 'C']	{'A': 0, 'B': 0, 'C': 1}

7.5 Bellman-Ford -- when weights can be negative

Dijkstra's finalization assumption breaks the moment a negative edge exists: a node that looked cheapest right now could become even cheaper through a later negative edge.



$$\text{dist}[C] = \text{dist}[B] + (-10) = 5 - 10 = -5$$

The fix is to stop trying to finalize nodes at all, and instead keep **relaxing every edge** -- checking whether `dist[u] + weight < dist[v]` -- for up to `V-1` passes. Why `V-1`? A shortest *simple* path (no repeated vertex) has at most `V-1` edges, since a path with `V` or more edges would have to repeat a vertex. Pass 1 finds all shortest paths using at most 1 edge, pass 2 extends that to at most 2 edges, and so on -- after `V-1` passes, every possible shortest simple path has been considered.

```
def bellman_ford(n, edges, start):
    """edges = [(u, v, weight)]. Returns None if a reachable negative
    cycle exists, else the distance array."""
    dist = [float("inf")] * n
    dist[start] = 0

    for _ in range(n - 1):
        changed = False
        for u, v, weight in edges:
            if dist[u] == float("inf"):
                continue
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
                changed = True
        if not changed:
            break

    for u, v, weight in edges:
        if dist[u] != float("inf") and dist[u] + weight < dist[v]:
            return None

    return dist
```

Full trace:

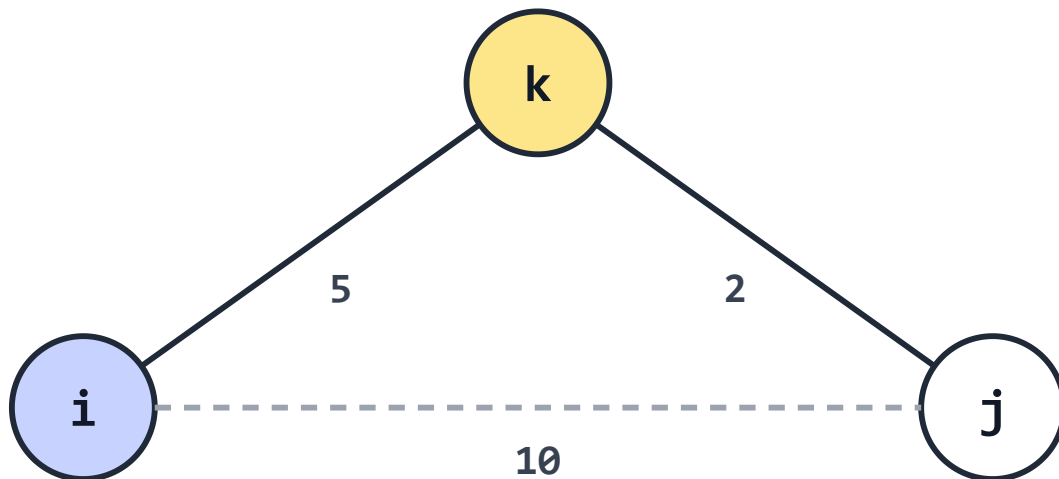
pass	edge relaxed	effect	dist[] after
start	-	-	{'A': 0, 'B': inf, 'C': inf}
pass 1	A->B	relax to 5	{'A': 0, 'B': 5, 'C': inf}
pass 1	B->C	relax to -5	{'A': 0, 'B': 5, 'C': -5}
pass 2	-	no change, could stop early	-

If a distance can *still* be improved on one extra pass beyond $V-1$, that's proof of a reachable **negative cycle** -- there's no bound on how low the distance could keep going, since the cycle can be looped forever. Complexity: $O(V \cdot E)$ time (much slower than Dijkstra's $O(E \log V)$ for sparse graphs), $O(V)$ space.

7.6 Floyd-Warshall -- all pairs at once

Every algorithm so far answers "shortest path from ONE source." When the question is "shortest path between **every** pair," Floyd-Warshall's different angle is more efficient than running Dijkstra from every node (though for sparse non-negative graphs, that's also a valid approach).

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$



$$\min(10, 5+2)=7: k \text{ gives a cheaper route}$$

The core question, asked for every intermediate node k : is going through k cheaper than the current best route from i to j ?

```
def floyd_warshall(dist):
    """dist is an n x n matrix, mutated and returned in place.
    dist[i][i] = 0, dist[i][j] = edge weight or inf if no edge."""
    n = len(dist)
    for k in range(n):
        for i in range(n):
            if dist[i][k] == float("inf"):
                continue
            for j in range(n):
                through_k = dist[i][k] + dist[k][j]
                if through_k < dist[i][j]:
                    dist[i][j] = through_k
    return dist
```

`k`, `i`, `j` as three nested loops means `dist[i][j]` gradually improves as more and more intermediate nodes are "allowed" to be used as stopovers, starting with just node 0, then nodes 0-1, then 0-1-2, and so on until every vertex has been considered as a possible stopover. Complexity: $O(V^3)$ time, $O(V^2)$ space -- fine for small/moderate `V`, but not for large sparse graphs.

7.7 Recap

Algorithm	Exploits	Data structure
BFS	equal edge costs	plain queue
0-1 BFS	costs in {0, 1}	deque
Dijkstra	non-negative weights	min-heap
Bellman-Ford	negative edges allowed	nothing special -- repeat and relax
Floyd-Warshall	needs ALL pairs	$O(V^2)$ DP table

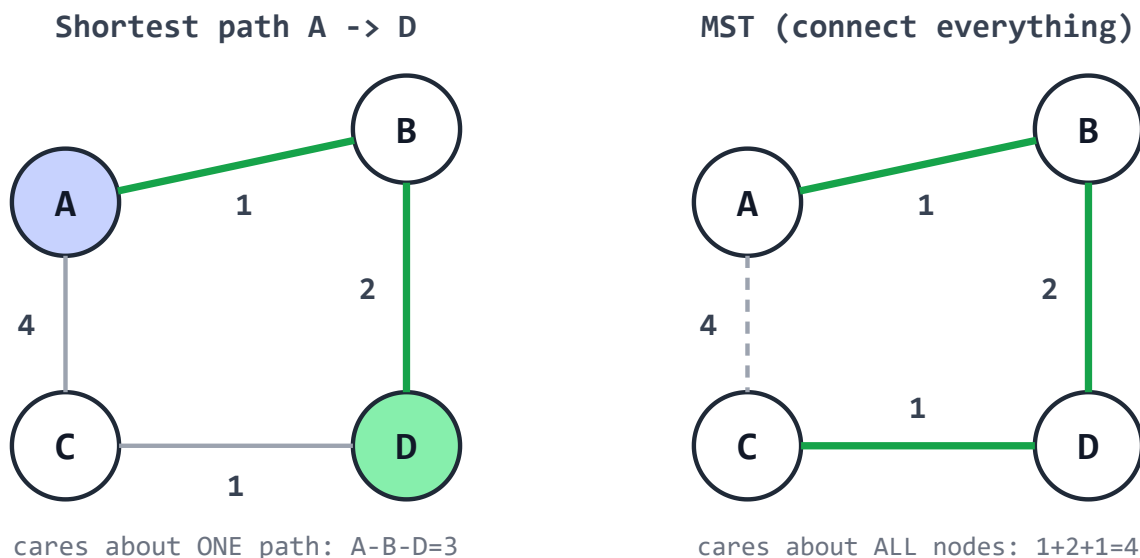
Part 8: Minimum Spanning Tree -- Prim vs. Kruskal

8.1 Spanning tree, then minimum

A spanning tree of a graph: includes every vertex, stays connected, and has no cycle -- which for V vertices always means exactly $V-1$ edges. A **minimum** spanning tree is the spanning tree whose total edge weight is smallest.

8.2 MST is not shortest path

This distinction matters more than it sounds like it should, because the two problems can look deceptively similar:

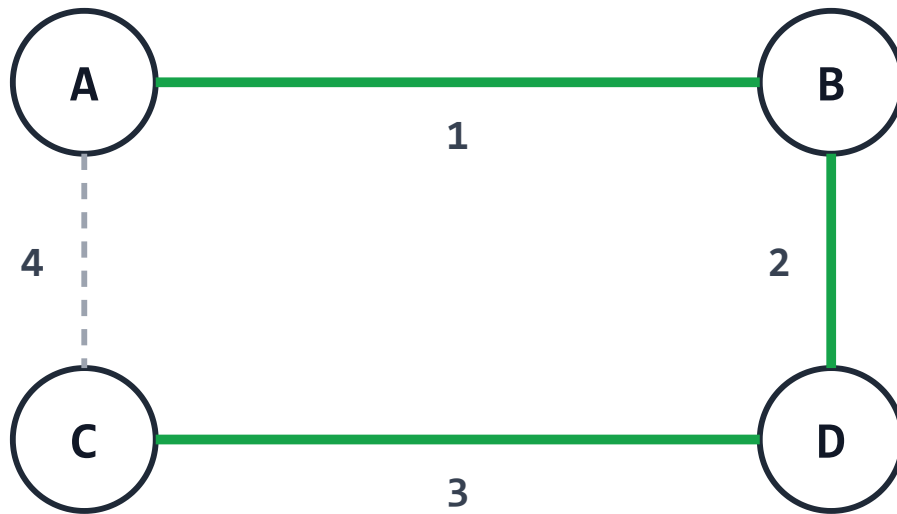


- **Shortest path** cares about *one* source and *one* destination.
- **MST** cares about connecting *every* vertex as cheaply as possible in total -- it may deliberately avoid the direct edge between two particular nodes if a different global arrangement is cheaper overall.

8.3 Kruskal -- think in terms of edges

Consider edges from cheapest to most expensive. Add an edge only if it connects two currently separate components.

Kruskal: cheapest edge first, skip if same component



took 1,2,3 (total 6); edge 4 would cycle

```
def kruskal(n, edges):  
    """MST total weight. edges = [(weight, u, v)]. -1 if disconnected."""  
    edges = sorted(edges)  
    dsu = DSU(n)  
    total = 0  
    count = 0  
    for weight, u, v in edges:  
        if dsu.union(u, v):  
            total += weight  
            count += 1  
            if count == n - 1:  
                break  
    return total if count == n - 1 else -1
```

Full trace:

edge	decision	running total	components left
A-B (w=1)	take	1	3
B-D (w=2)	take	3	2
C-D (w=3)	take	6	1
A-C (w=4)	skip (cycle)	6	1

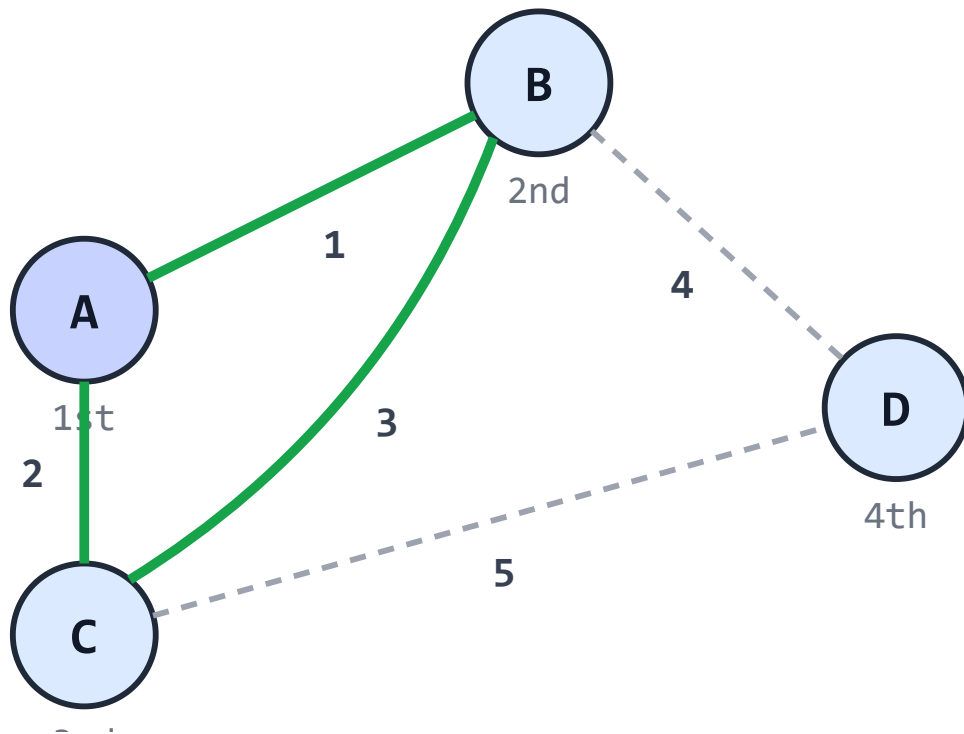
Because the core question at every step is "does this edge connect two different components," Kruskal pairs naturally with DSU: `find(a) == find(b)` means adding the edge would only create

a cycle, so skip it.

8.4 Prim -- think in terms of the growing tree

Grow one connected tree by repeatedly taking the cheapest edge from the tree to an unvisited vertex.

Prim: grow one tree, cheapest boundary edge



```
def prim(n, graph):
    """MST total weight. graph[u] = [(v, weight), ...]. -1 if disconnected."""
    visited = [False] * n
    heap = [(0, 0)]
    total = 0
    count = 0

    while heap:
        weight, node = heapq.heappop(heap)
        if visited[node]:
            continue
        visited[node] = True
        total += weight
        count += 1
        for nei, edge_weight in graph[node]:
            if not visited[nei]:
                heapq.heappush(heap, (edge_weight, nei))

    return total if count == n else -1
```

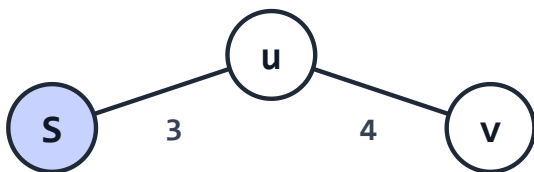
The `if visited[node]: continue` guard matters: the heap can hold several candidate edges into the same not-yet-visited vertex (pushed at different times from different tree members), and only the first, cheapest one that actually gets popped should be accepted -- the rest are stale.

8.5 Dijkstra vs. Prim -- a common interview confusion

Their code looks almost identical (both use a min-heap over `(value, node)` pairs), but the *meaning* of the heap value is completely different:

Dijkstra

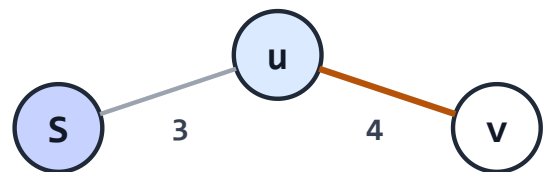
heap priority =
TOTAL distance from source



$\text{priority}(v) = 3 + 4 = 7$ (path total)

Prim

heap priority =
the NEXT connecting edge alone



$\text{priority}(v) = 4$ (just this edge)

- **Dijkstra**: heap priority = **total distance from the source** so far along this path.
- **Prim**: heap priority = **just the weight of the one edge** that would connect this candidate node to the growing tree, nothing accumulated.

8.6 Comparison table and the cut property

	Prim	Kruskal
Thinks in terms of	vertices (one growing tree)	edges (globally)
Main tool	min-heap	sorted edges + DSU
Needs a global sort?	no	yes
Cycle handling	<code>visited[]</code> array	DSU
Natural input shape	adjacency list	edge list

Both rely on the **cut property**: split the vertices into any two groups, and among all edges crossing that split, the cheapest one can always be safely added to an MST. Prim repeatedly creates such a cut (current tree vs. everything else); Kruskal implicitly respects cuts created by its current set of connected components.

8.7 Modeling tricks: complete graphs and virtual sources

Min Cost to Connect All Points (LC 1584) has an *implicit* complete graph (every pair of points can connect), so building all $O(n^2)$ edges explicitly and running Kruskal is wasteful -- Prim naturally avoids ever materializing the full edge list:

```
def min_cost_connect_points(points):
    """LC 1584 -- Prim's algorithm on an implicit complete graph."""
    n = len(points)
    visited = [False] * n
    heap = [(0, 0)]
    total = 0
    count = 0

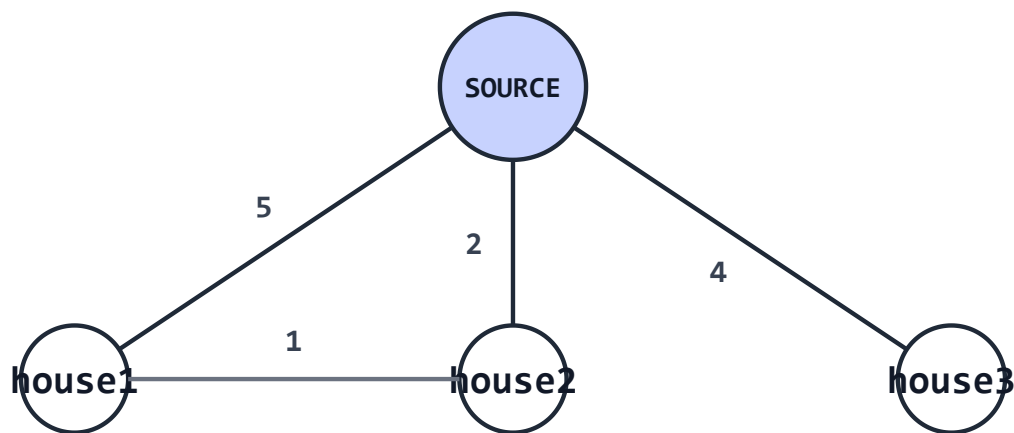
    while heap and count < n:
        cost, node = heapq.heappop(heap)
        if visited[node]:
            continue
        visited[node] = True
        total += cost
        count += 1
        x1, y1 = points[node]
        for nei in range(n):
            if visited[nei]:
                continue
            x2, y2 = points[nei]
            heapq.heappush(heap, (abs(x1 - x2) + abs(y1 - y2), nei))

    return total
```

Connecting Cities With Minimum Cost (LC 1135) is the same Kruskal pattern applied directly to a given edge list:

```
def minimum_cost_connecting(n, connections):
    """LC 1135 Connecting Cities With Minimum Cost -- Kruskal + DSU.
    connections = [(u, v, cost)], nodes are 1..n."""
    connections = sorted(connections, key=lambda x: x[2])
    dsu = DSU(n + 1)
    total = 0
    edges_used = 0
    for u, v, cost in connections:
        if dsu.union(u, v):
            total += cost
            edges_used += 1
            if edges_used == n - 1:
                return total
    return -1
```

Optimize Water Distribution in a Village (LC 1168) needs a genuine modeling insight: each house can *either* build its own well (a fixed cost) *or* connect to another house via a pipe. That "well" option doesn't look like a graph edge at first, but it becomes one with a virtual source trick:



well cost = edge to SOURCE; now it's ordinary MST (run Kruskal)

Create a virtual SOURCE node. SOURCE → house[i] with weight wells[i] -- now "build a well" and "lay a pipe" are both just edges, and the whole problem is ordinary MST.

```

def min_cost_to_supply_water(n, wells, pipes):
    """LC 1168 -- virtual SOURCE node (0) trick: connecting to the source
    at cost wells[i] is the same as building a well; then it's plain MST."""
    edges = []
    for house in range(1, n + 1):
        edges.append((wells[house - 1], 0, house))
    for u, v, cost in pipes:
        edges.append((cost, u, v))
    edges.sort()

    dsu = DSU(n + 1)
    total = 0
    for cost, u, v in edges:
        if dsu.union(u, v):
            total += cost
    return total

```

Recognition for the whole MST family: "connect all nodes/points with minimum total cost" -> MST -> Kruskal if you naturally have (or can build) an edge list, Prim if you naturally have an adjacency list and want to grow from a start.

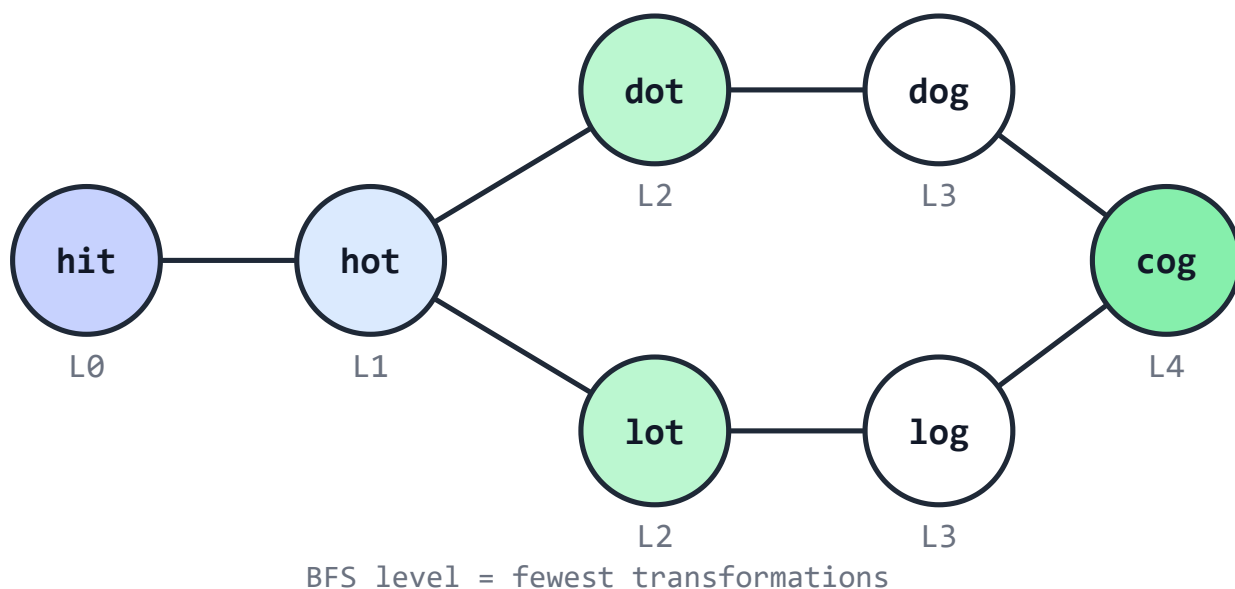
Part 9: Hidden-State Graphs

9.1 Word Ladder -- the graph isn't given, it's implied

This is one of the most important interview patterns because the input is just two words and a dictionary -- no edges anywhere in sight.

Each word is a node. Two words have an edge if they differ by exactly one character.

`hit -> cog: each word is a node, edges = 1 letter apart`



Once that model clicks, the rest follows automatically: it's a shortest path in an unweighted graph (every transformation costs 1), so BFS.

```
def ladder_length(begin_word, end_word, word_list):
    """LC 127 Word Ladder -- each word is a node, an edge = differ by
    exactly one character. Unweighted shortest path → BFS."""
    words = set(word_list)
    if end_word not in words:
        return 0

    queue = deque([(begin_word, 1)])
    visited = {begin_word}

    while queue:
        word, steps = queue.popleft()
        if word == end_word:
            return steps
        for i in range(len(word)):
            for ch in "abcdefghijklmnopqrstuvwxyz":
                new_word = word[:i] + ch + word[i + 1:]
                if new_word in words and new_word not in visited:
                    visited.add(new_word)
                    queue.append((new_word, steps + 1))

    return 0
```

Full trace:

word popped	BFS level (steps)	visited so far
hit	1	{hit}
hot	2	{hit,hot}
dot	3	{dot,hit,hot,lot}
lot	3	{dog,dot,hit,hot,lot}
dog	4	{dog,dot,hit,hot,log,lot}
log	4	{cog,dog,dot,hit,hot,log,lot}
cog	5	{cog,dog,dot,hit,hot,log,lot}

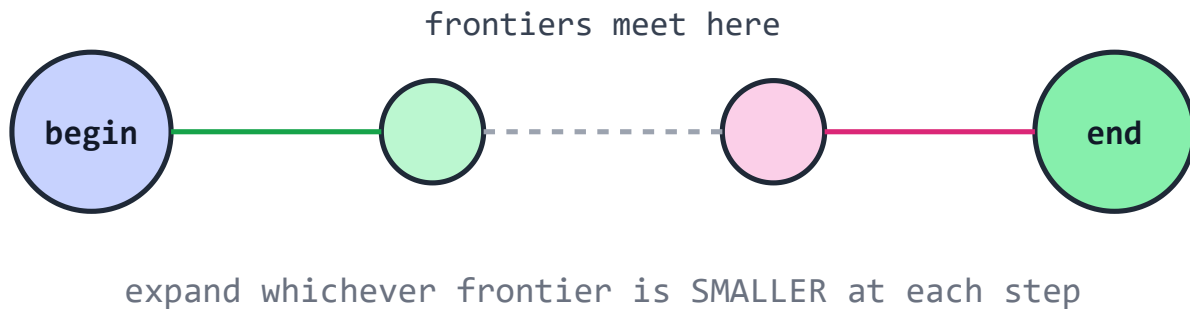
The general recipe for *any* "transform X into Y" problem:

1. What is a **state**? (here: a word)
2. When are two states **connected**? (differ by exactly one character)
3. Does each transition cost the same? If yes -> BFS.

This generalizes far past word ladders: a sliding puzzle's state is the board layout, rotting oranges' state is a grid cell, a combination lock's state is the digit string, a knight's-move problem's state is a square on the board. All of them collapse to: **STATE → NEIGHBORS → GRAPH → BFS/DFS/Dijkstra** .

9.2 Bidirectional BFS

When both the start and target are known in advance and the branching factor is large, searching from **both ends simultaneously** and always expanding whichever frontier is currently smaller can dramatically shrink the search space.



```
def ladder_length_bidirectional(begin_word, end_word, word_list):
    """Same problem, bidirectional BFS -- always expand the smaller
    frontier so the search space grows much more slowly."""
    words = set(word_list)
    if end_word not in words:
        return 0

    begin, end = {begin_word}, {end_word}
    visited = {begin_word, end_word}
    steps = 1

    while begin and end:
        if len(begin) > len(end):
            begin, end = end, begin

        next_level = set()
        for word in begin:
            for i in range(len(word)):
                for ch in "abcdefghijklmnopqrstuvwxyz":
                    new_word = word[:i] + ch + word[i + 1:]
                    if new_word in end:
                        return steps + 1
                    if new_word in words and new_word not in visited:
                        visited.add(new_word)
                        next_level.add(new_word)

        begin = next_level
        steps += 1

    return 0
```

`test_sol.py` verifies this returns the identical answer to the single-direction BFS version on every test case -- it's a performance optimization, not a different algorithm.

9.3 More hidden-graph problems

The Maze (LC 490): the ball doesn't move one cell at a time -- it rolls until it hits a wall. So each BFS "edge" is a full roll-to-a-stop, not a single step, which is easy to miss if you copy a generic grid-BFS template without re-deriving the transition rule for this problem.

```
def has_path_maze(maze, start, destination):
    """LC 490 The Maze -- the ball rolls until it hits a wall; each
    'roll to a stop' is one BFS edge, not each single cell step."""
    rows, cols = len(maze), len(maze[0])
    visited = [[False] * cols for _ in range(rows)]
    queue = deque([tuple(start)])
    visited[start[0]][start[1]] = True

    while queue:
        r, c = queue.popleft()
        if [r, c] == list(destination):
            return True
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr, nc = r, c
            while (0 ≤ nr + dr < rows and 0 ≤ nc + dc < cols
                  and maze[nr + dr][nc + dc] == 0):
                nr += dr
                nc += dc
            if not visited[nr][nc]:
                visited[nr][nc] = True
                queue.append((nr, nc))

    return False
```

Snakes and Ladders (LC 909): the hidden graph lives in the board's 1D numbering. Square `i` connects to `i+1 .. i+6` (the six dice outcomes), except a square with a snake or ladder redirects straight to its destination instead. BFS distance = fewest dice rolls.

```

def snakes_and_ladders(board):
    """LC 909 -- the board's 1..n*n boustrophedon numbering is a hidden
    graph: square i has an edge to i+1..i+6 (dice), or straight to the
    destination if a snake/ladder is there. BFS = fewest dice rolls."""
    n = len(board)

    def cell(num):
        # num is 1-indexed; convert to (row, col) in the boustrophedon layout
        num -= 1
        row = num // n
        col = num % n
        if row % 2 == 1:
            col = n - 1 - col
        r = n - 1 - row
        return r, col

    target = n * n
    dist = {1: 0}
    queue = deque([1])

    while queue:
        num = queue.popleft()
        if num == target:
            return dist[num]
        for nxt in range(num + 1, min(num + 6, target) + 1):
            r, c = cell(nxt)
            dest = nxt if board[r][c] == -1 else board[r][c]
            if dest not in dist:
                dist[dest] = dist[num] + 1
                queue.append(dest)

    return -1

```

Nearest Exit from Entrance in Maze (LC 1926): plain grid BFS, with one easy-to-miss rule -- the entrance cell itself is never counted as a valid exit, even though it sits on the border like a real exit would.

```

def nearest_exit(maze, entrance):
    """LC 1926 -- plain grid BFS; the entrance cell itself is never a
    valid exit even though it's a border cell."""
    rows, cols = len(maze), len(maze[0])
    sr, sc = entrance
    visited = [[False] * cols for _ in range(rows)]
    visited[sr][sc] = True
    queue = deque([(sr, sc, 0)])

    while queue:
        r, c, steps = queue.popleft()
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr, nc = r + dr, c + dc
            in_bounds = 0 ≤ nr < rows and 0 ≤ nc < cols
            if in_bounds and not visited[nr][nc] and maze[nr][nc] == '.':
                if nr == 0 or nr == rows - 1 or nc == 0 or nc == cols - 1:
                    return steps + 1
                visited[nr][nc] = True
                queue.append((nr, nc, steps + 1))

    return -1

```

Recognition for the whole family: a problem "doesn't look like a graph" exactly when its nodes are some other kind of state (a word, a board position, a maze cell after a roll) -- the fix is always the same three questions from 9.1, not a different algorithm.

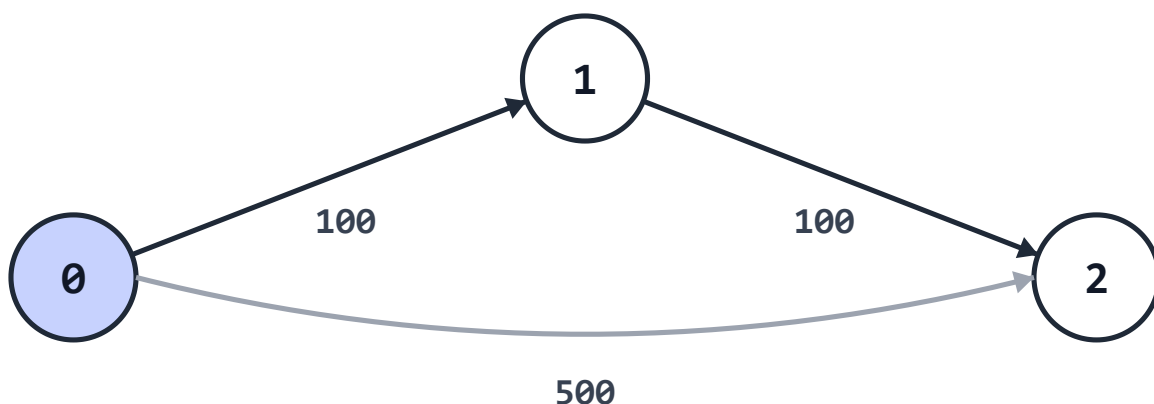
Part 10: Constrained Shortest Paths -- Expanding the State

10.1 Cheapest Flights Within K Stops

This problem looks like Dijkstra at first (non-negative weights, shortest path), but there's an extra constraint: **at most K stops**.

The key realization: `dist[node]` alone is no longer enough information. A cheaper route that uses too many stops is unusable, so the state has to become `(node, number_of_edges_used)`.

`K=1 stop -> at most K+1 = 2 edges (2 relaxation rounds)`



round1: `dist[1]=100, dist[2]=500`

round2: `dist[2]=min(500, 200)=200`

That reframing turns this into a Bellman-Ford variant: `K` stops means at most `K+1` edges, which means exactly `K+1` relaxation rounds.

```
def find_cheapest_price(n, flights, src, dst, k):
    """LC 787 -- Bellman-Ford with exactly k+1 relaxation rounds; must
    copy dist each round so a round only uses last round's information
    (otherwise a single round could chain more than one new edge)."""
    INF = float("inf")
    dist = [INF] * n
    dist[src] = 0

    for _ in range(k + 1):
        new_dist = dist.copy()
        for u, v, price in flights:
            if dist[u] == INF:
                continue
            new_dist[v] = min(new_dist[v], dist[u] + price)
        dist = new_dist

    return -1 if dist[dst] == INF else dist[dst]
```

The critical implementation detail: `new_dist = dist.copy()` each round. If distances were updated in place, an improvement made earlier in the *same* round could immediately be used by a later edge in that same round -- silently letting a path use more than one additional edge per round and breaking the "K+1 edges max" guarantee. Copying ensures each round only ever builds on the *previous* round's results.

Recognition: "cheapest/shortest path" + "at most K edges/stops/moves" -> the number of edges used needs to become part of the state -> Bellman-Ford-style layered relaxation (or a modified Dijkstra tracking `dist[node][stops]`).

10.2 The general "expand the state" pattern

The Cheapest Flights insight generalizes far beyond stop counts. Whenever a constraint (a coupon, a fuel budget, a color you just used, a key you've picked up) affects which future moves are legal or cheap, fold that constraint directly into the state Dijkstra searches over:

```

def shortest_with_one_coupon(graph, start, target, n=None):
    """Generic 'expand the state' template: state = (node, coupon_used).
    A coupon halves the cost of the edge it's used on."""
    if n is None:
        n = len(graph)
    dist = [[float("inf"), float("inf")] for _ in range(n)]
    dist[start][0] = 0
    heap = [(0, start, 0)]

    while heap:
        cost, node, used = heapq.heappop(heap)
        if cost > dist[node][used]:
            continue
        for nei, weight in graph[node]:
            new_cost = cost + weight
            if new_cost < dist[nei][used]:
                dist[nei][used] = new_cost
                heapq.heappush(heap, (new_cost, nei, used))
            if used == 0:
                half_cost = cost + weight // 2
                if half_cost < dist[nei][1]:
                    dist[nei][1] = half_cost
                    heapq.heappush(heap, (half_cost, nei, 1))

    return min(dist[target])

```

Normal shortest path: `dist[node]` . Constrained shortest path: `dist[node][state]` , where state might be stops used, fuel remaining, or a coupon already spent.

10.3 Shortest Path with Alternating Colors (LC 1129)

A direct application of the same idea: you're not allowed to use two edges of the same color consecutively, so the state has to remember which color was used last.

```

def shortest_alternating_paths(n, red_edges, blue_edges):
    """LC 1129 -- state = (node, last_color_used). BFS since every edge
    costs 1 regardless of color."""
    RED, BLUE = 0, 1
    graph = [[] for _ in range(n)] # graph[u][RED]/[BLUE] = neighbors
    for u, v in red_edges:
        graph[u][RED].append(v)
    for u, v in blue_edges:
        graph[u][BLUE].append(v)

    dist = [[-1, -1] for _ in range(n)]
    dist[0][RED] = dist[0][BLUE] = 0
    queue = deque([(0, RED), (0, BLUE)])

    while queue:
        node, color = queue.popleft()
        next_color = 1 - color
        for nei in graph[node][next_color]:
            if dist[nei][next_color] == -1:
                dist[nei][next_color] = dist[node][color] + 1
                queue.append((nei, next_color))

    result = []
    for i in range(n):
        best = [d for d in dist[i] if d != -1]
        result.append(min(best) if best else -1)
    return result

```

Every edge still costs 1, so it's still BFS -- just BFS over the expanded state `(node, last_color)` instead of plain `node`, with two starting states `(0, RED)` and `(0, BLUE)` since the very first move is unconstrained.

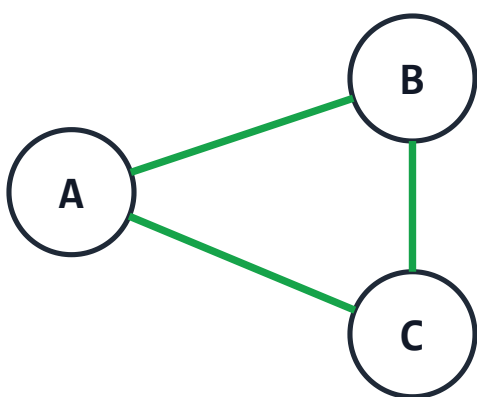
Recognition: if the problem's wording implies "you can't do the same thing twice in a row" or "you have a limited resource that unlocks a special move," that's a signal to widen the state, not to invent a new algorithm.

Part 11: Eulerian Paths -- Reconstruct Itinerary

11.1 Eulerian vs. Hamiltonian

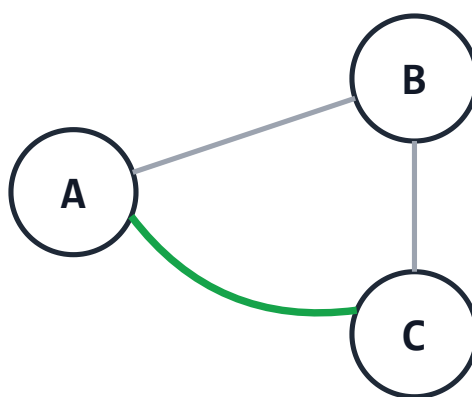
Reconstruct Itinerary looks like a DFS problem ("visit airports using tickets"), but the real requirement is different from anything covered so far: **use every ticket (edge) exactly once**, not every airport (node) exactly once.

Eulerian: every **EDGE** once



A-B-C-A uses each edge once

Hamiltonian: every **NODE** once



A-C-B visits each node once

Eulerian -> every **EDGE** exactly once. Hamiltonian -> every **VERTEX** exactly once.

11.2 Why greedy DFS alone doesn't work

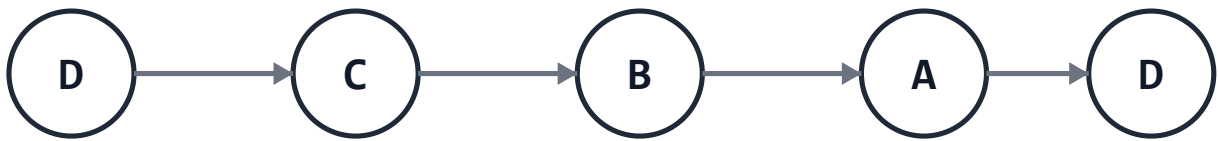
A naive DFS that always picks the smallest next destination can leave an edge "stranded" -- reachable from nowhere once the rest of the graph has already been exhausted along a different branch. The fix requires **not finalizing a node the moment we enter it**:

Finalize a node only when it can no longer leave (no unused outgoing edges remain).

11.3 Hierholzer's algorithm

Finalize a node only when it has NO unused outgoing edges left

append order (postorder): D, C, B, A, D



reverse it -> the real itinerary: D, A, B, C, D

The result is built in **postorder** (a node is appended only after every one of its outgoing edges has been consumed) and then reversed -- the exact same "compute postorder, then reverse" idea as the DFS-based topological sort in Part 4.5.

```
def find_itinerary(tickets):
    """LC 332 -- Hierholzer's algorithm. Use every EDGE (ticket) exactly
    once; a node is finalized (appended to result) only when it has no
    unused outgoing edges left, and the whole result is reversed."""
    graph = defaultdict(list)
    for src, dst in tickets:
        graph[src].append(dst)
    for src in graph:
        graph[src].sort(reverse=True) # so .pop() gives smallest first

    result = []

    def dfs(node):
        while graph[node]:
            dfs(graph[node].pop())
        result.append(node)

    dfs("JFK")
    return result[::-1]
```

Reverse-sorting each node's destination list (`sort(reverse=True)`) so `.pop()` removes the smallest-lexicographic option in `O(1)` is a small but important efficiency detail -- `pop(0)` on a normal list is `O(n)`.

Recognition: "use every edge/ticket/domino exactly once" -> Eulerian path -> Hierholzer's.

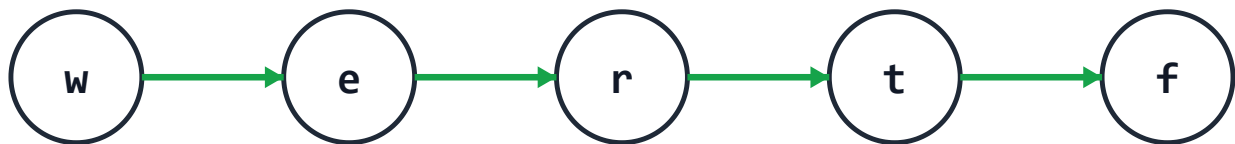
Complexity: `O(E log E)` (dominated by sorting destinations), `O(E + V)` space.

Part 12: Alien Dictionary -- Hidden Graph + Topological Sort

12.1 Inferring edges from adjacent words

The input is just a sorted word list -- no explicit ordering constraints anywhere. The graph has to be **inferred**: compare each pair of adjacent words, find their first differing character, and that gives one ordering edge.

inferred from first differing letters -> topological sort



For "wrt" vs "wrf", the first difference is **t** vs **f**, giving **t → f**. Chaining through the whole list (wrt/wrf , wrf/er , er/ett , ett/rftt) builds up **w → e → r → t → f**.

12.2 Why only adjacent words matter

Comparing every pair of words (not just neighbors) would create a lot of redundant edges -- if the dictionary already guarantees **A < B < C** globally through consecutive comparisons, comparing **A** directly against **C** adds no new information topological sort doesn't already have via **A < B** and **B < C**.

12.3 Two invalid cases

- **Prefix violation:** ["abc", "ab"] -- normally a shorter prefix sorts first ("ab" < "abc"), so seeing the longer word appear *before* its own prefix is a direct contradiction. Must be checked explicitly before scanning for the first differing character (there may not be one, since one word is a prefix of the other).
- **Cycle:** if the inferred edges eventually loop back on themselves (**a → b → c → a**), there's no valid ordering -- exactly the same "Kahn's algorithm can't process every node" signal from Part 4.4.

12.4 Code

```

def alien_order(words):
    """LC 269 -- infer character-ordering edges from adjacent words'
    first difference, then Kahn's algorithm on the alphabet."""
    graph = defaultdict(set)
    indegree = {ch: 0 for word in words for ch in word}

    for i in range(len(words) - 1):
        w1, w2 = words[i], words[i + 1]
        if len(w1) > len(w2) and w1.startswith(w2):
            return "" # invalid: longer word can't be a prefix-predecessor
        for j in range(min(len(w1), len(w2))):
            if w1[j] != w2[j]:
                if w2[j] not in graph[w1[j]]:
                    graph[w1[j]].add(w2[j])
                    indegree[w2[j]] += 1
                break

    queue = deque(ch for ch in indegree if indegree[ch] == 0)
    result = []
    while queue:
        ch = queue.popleft()
        result.append(ch)
        for nei in graph[ch]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                queue.append(nei)

    return "".join(result) if len(result) == len(indegree) else ""

```

Structurally, this is three stages: build one node per character seen, build one edge per adjacent-word first-difference, then run ordinary Kahn's-algorithm topological sort over the resulting alphabet graph.

Recognition: words / ordering / precedence / before-after / prerequisites in string form -> ask "can I convert these relationships into directed edges?" -> if yes, it's topological sort wearing a string problem's clothes. This exact "hidden dependency graph" shape also shows up as Course Schedule, build-system ordering, and task scheduling -- the graph pattern doesn't change even though the surface problem does.

One more subtlety worth remembering: topological sort doesn't guarantee a *unique* answer (Part 4.2's correction is the reminder). If a problem specifically asks for the lexicographically smallest valid ordering, swap the plain queue for a **min-heap** of zero-indegree nodes instead.

Part 13: Hard and Mixed Problems

13.1 Network Delay Time -- Dijkstra, read carefully

A direct Dijkstra application with one twist in what the *answer* means: run Dijkstra from the source `k`, then the time for the signal to reach everyone is the **maximum** finite distance across all nodes (the last node to hear about it), and if any node is unreachable, the answer is -1.

```
def network_delay_time(times, n, k):
    """LC 743 -- Dijkstra from k; answer = max finite distance, or -1."""
    graph = [[] for _ in range(n + 1)]
    for u, v, w in times:
        graph[u].append((v, w))

    dist = dijkstra(graph, k, n + 1)
    answer = max(dist[1:])
    return -1 if answer == float("inf") else answer
```

13.2 Minimax Dijkstra -- Path With Minimum Effort, Swim in Rising Water

Both of these replace the usual "sum of edge weights" cost with "**maximum edge weight along the path**" -- a genuinely different objective, but Dijkstra's greedy structure still works because the relaxation check just changes shape.

Path cost = MAX edge, not the sum

1	2	2
3	8	2
5	3	5

heap key = running max effort

```

def minimum_effort_path(heights):
    """LC 1631 -- Dijkstra where 'distance' is the MAX edge on the path,
    not the sum (a minimax shortest path)."""
    rows, cols = len(heights), len(heights[0])
    dist = [[float("inf")] * cols for _ in range(rows)]
    dist[0][0] = 0
    heap = [(0, 0, 0)]
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while heap:
        effort, r, c = heapq.heappop(heap)
        if (r, c) == (rows - 1, cols - 1):
            return effort
        if effort > dist[r][c]:
            continue
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols:
                edge = abs(heights[r][c] - heights[nr][nc])
                new_effort = max(effort, edge)
                if new_effort < dist[nr][nc]:
                    dist[nr][nc] = new_effort
                    heapq.heappush(heap, (new_effort, nr, nc))

    return 0

```

`new_effort = max(effort, edge)` instead of `effort + edge` is the only real change from ordinary Dijkstra -- everything else (min-heap, "pop the smallest, relax neighbors, skip stale entries") is identical.

```

def swim_in_water(grid):
    """LC 778 -- same minimax-Dijkstra idea as minimum_effort_path."""
    n = len(grid)
    visited = [[False] * n for _ in range(n)]
    heap = [(grid[0][0], 0, 0)]
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while heap:
        time, r, c = heapq.heappop(heap)
        if visited[r][c]:
            continue
        visited[r][c] = True
        if (r, c) == (n - 1, n - 1):
            return time
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < n and 0 ≤ nc < n and not visited[nr][nc]:
                heapq.heappush(heap, (max(time, grid[nr][nc]), nr, nc))

    return -1

```

13.3 Number of Ways to Arrive at Destination

A Dijkstra + counting hybrid: alongside `dist[node]`, maintain `ways[node]` (number of distinct shortest paths reaching that node). When a strictly shorter route is found, *replace* the count; when an *equally* short route is found, *add* to it.

```
def count_paths(n, roads):
    """LC 1976 Number of Ways to Arrive at Destination -- Dijkstra, plus
    a DP-style count of how many shortest paths reach each node."""
    MOD = 10 ** 9 + 7
    graph = [[] for _ in range(n)]
    for u, v, w in roads:
        graph[u].append((v, w))
        graph[v].append((u, w))

    dist = [float("inf")] * n
    ways = [0] * n
    dist[0] = 0
    ways[0] = 1
    heap = [(0, 0)]

    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]:
            continue
        for nei, w in graph[node]:
            nd = d + w
            if nd < dist[nei]:
                dist[nei] = nd
                ways[nei] = ways[node]
                heapq.heappush(heap, (nd, nei))
            elif nd == dist[nei]:
                ways[nei] = (ways[nei] + ways[node]) % MOD

    return ways[n - 1] % MOD
```

13.4 Find Eventual Safe States -- three-color DFS again

A node is "safe" if every path starting from it eventually terminates (reaches a node with no outgoing edges) without ever looping. This is the three-color DFS from Part 4.4 applied to a slightly different question: instead of just detecting *whether* a cycle exists anywhere, it determines *which specific nodes* can reach one.

```

def find_eventual_safe_states(graph):
    """LC 802 -- a node is 'safe' iff every path from it eventually
    terminates (hits a node with no outgoing edges) without looping.
    Three-color DFS: a node is unsafe iff it can reach a node on the
    current recursion stack (a cycle)."""
    n = len(graph)
    WHITE, GRAY, BLACK = 0, 1, 2
    state = [WHITE] * n

    def dfs(node):
        if state[node] != WHITE:
            return state[node] == BLACK
        state[node] = GRAY
        for nei in graph[node]:
            if not dfs(nei):
                return False
        state[node] = BLACK
        return True

    return [node for node in range(n) if dfs(node)]

```

A node is unsafe exactly when it can reach a GRAY node (one currently on its own recursion stack) -
 - the same cycle signal as directed cycle detection, just recorded per-node instead of as one global
 yes/no.

13.5 Course Schedule IV -- transitive closure

"Is course A a prerequisite of course B, possibly transitively?" is a reachability question across many
 queries -- with course counts small enough (≤ 100) that a Floyd-Warshall-style boolean transitive
 closure is simple and fast enough, rather than re-running a fresh BFS per query.

```

def check_if_prerequisite(num_courses, prerequisites, queries):
    """LC 1462 Course Schedule IV -- transitive-closure reachability.
    n is small ( $\leq 100$ ) so Floyd-Warshall-style boolean closure is simple
    and fast enough."""
    reach = [[False] * num_courses for _ in range(num_courses)]
    for u, v in prerequisites:
        reach[u][v] = True

    for k in range(num_courses):
        for i in range(num_courses):
            if reach[i][k]:
                for j in range(num_courses):
                    if reach[k][j]:
                        reach[i][j] = True

    return [reach[u][v] for u, v in queries]

```

13.6 Find All Possible Recipes -- Kahn's, ingredients as prerequisites

A recipe can only be made once every one of its ingredients is available -- either a raw supply, or another recipe that's already been made. That's exactly indegree: a recipe's indegree is the count of its ingredients that are themselves *other recipes* (raw supplies start already satisfied, contributing 0 to indegree).

```
def find_all_recipes(recipes, ingredients, supplies):
    """LC 2115 -- Kahn's algorithm: a recipe's indegree = number of its
    ingredients that are themselves recipes (raw supplies start satisfied)."""
    supply_set = set(supplies)
    graph = defaultdict(list)  # ingredient → recipes that need it
    indegree = {}

    for recipe, needed in zip(recipes, ingredients):
        indegree[recipe] = 0
        for ing in needed:
            if ing not in supply_set:
                graph[ing].append(recipe)
                indegree[recipe] += 1

    queue = deque(r for r in recipes if indegree[r] == 0)
    made = set()
    while queue:
        recipe = queue.popleft()
        made.add(recipe)
        for nxt in graph[recipe]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)

    return [r for r in recipes if r in made]
```

13.7 Reorder Routes -- direction-aware DFS on an undirected traversal

Every city must be able to reach city 0, but the roads are directed and were built without that goal in mind. Traverse the underlying **undirected** tree from city 0, but remember each edge's *original* direction: an edge that originally pointed **away** from city 0 (in the direction DFS is walking) needs to be reversed; one that already points **toward** city 0 is free.

```

def min_reorder(n, connections):
    """LC 1466 -- treat roads as undirected for traversal but remember
    which direction each edge originally pointed; DFS from city 0 and
    count edges that point 'toward' 0 instead of away from it (those
    need to be flipped)."""
    graph = [[] for _ in range(n)]
    for u, v in connections:
        graph[u].append((v, 1))    # u→v is the original direction: flip it
        graph[v].append((u, 0))    # v→u is already the correct direction

    visited = [False] * n
    changes = 0

    def dfs(node):
        nonlocal changes
        visited[node] = True
        for nei, cost in graph[node]:
            if not visited[nei]:
                changes += cost
                dfs(nei)

    dfs(0)
    return changes

```

13.8 Evaluate Division -- weighted directed graph, query = path product

$a / b = 2.0$ becomes a directed edge $a \rightarrow b$ with weight 2.0 , and *also* the reverse edge $b \rightarrow a$ with weight 0.5 (division is invertible). A query x / y is then "walk from x to y , multiplying edge weights along the way" -- if no path exists, the answer is -1.0 .

```

def calc_equation(equations, values, queries):
    """LC 399 Evaluate Division -- build a weighted DIRECTED graph where
    edge a→b has weight (a/b); answer a query x/y via DFS multiplying
    edge weights along a path from x to y."""
    graph = defaultdict(dict)
    for (a, b), val in zip(equations, values):
        graph[a][b] = val
        graph[b][a] = 1.0 / val

    def dfs(node, target, visited):
        if node not in graph or target not in graph:
            return -1.0
        if node == target:
            return 1.0
        visited.add(node)
        for nei, weight in graph[node].items():
            if nei not in visited:
                result = dfs(nei, target, visited)
                if result != -1.0:
                    return weight * result
        return -1.0

    return [dfs(x, y, set()) for x, y in queries]

```

13.9 Two quick ones: Find Center of Star Graph, All Ancestors

Find Center of Star Graph (LC 1791): in a star graph every edge touches the center, so the center must be one of the two endpoints of the very first edge -- checking which of those two also appears in the second edge resolves the (rare) ambiguity when there are only two nodes.

```

def find_center(edges):
    """LC 1791 -- in a star graph the center appears in every edge, so
    it must be one of the two endpoints of the first edge (and, unless
    n==2, it's whichever of those two also appears in the second edge)."""
    a, b = edges[0]
    c, d = edges[1]
    return a if a in (c, d) else b

```

All Ancestors of a Node in a DAG (LC 2192): process nodes in topological order, propagating each node's ancestor set forward to its direct successors (a node's ancestors = union of its direct predecessors, plus each of *their* ancestor sets).

```

def get_ancestors(n, edges):
    """LC 2192 -- topological order + propagate ancestor sets forward
    (a node's ancestors = union of its direct predecessors' ancestor
    sets plus the predecessors themselves)."""
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    ancestor_sets = [set() for _ in range(n)]
    queue = deque(i for i in range(n) if indegree[i] == 0)

    while queue:
        node = queue.popleft()
        for nei in graph[node]:
            before = len(ancestor_sets[nei])
            ancestor_sets[nei].add(node)
            ancestor_sets[nei].update(ancestor_sets[node])
            indegree[nei] -= 1
            if indegree[nei] == 0:
                queue.append(nei)

    return [sorted(s) for s in ancestor_sets]

```

13.10 Number of Islands, once more, plainly

Worth including precisely because it's *not* hard -- flood-fill DFS is the cleanest possible illustration that grid = graph:

```

def num_islands(grid):
    """LC 200 -- DFS flood fill; grid = [['1', '0', ...], ...]."""
    rows, cols = len(grid), len(grid[0])
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    def dfs(r, c):
        if r < 0 or r ≥ rows or c < 0 or c ≥ cols or grid[r][c] ≠ "1":
            return
        grid[r][c] = "0"
        for dr, dc in directions:
            dfs(r + dr, c + dc)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                dfs(r, c)
    return count

```

Part 14: Recognition Patterns and the Final Decision Guide

14.1 Per-pattern recognition tables

Graph recognition -- don't wait for the word "graph":

Wording	Model
cities + roads	nodes = cities, edges = roads
people + friendships/enemies	nodes = people, edges = relationships
courses + prerequisites	nodes = courses, directed edges
grid + valid moves	nodes = cells, edges = neighbors
words + one-letter transforms	nodes = words, edges = 1-char diff
accounts + shared emails	nodes = emails, edges = "same account"

DFS recognition: reachability, counting components, flood fill, collecting all nodes of one component, undirected cycle detection (with parent tracking).

BFS recognition: "minimum/fewest steps/moves/edges" in an **unweighted** setting, level-order needs, multi-source spread (rotting oranges-style), shortest path reconstruction.

Cycle detection recognition: undirected -> DFS + parent, OR `edges > n - components`.
Directed -> Kahn's (can't process every node) OR three-color DFS (hits a GRAY node).

Topological sort recognition: "prerequisite", "dependency", "before", "build order", "scheduling" -> directed graph -> Kahn's or DFS postorder. Remember: **multiple valid orderings usually exist** (Part 4.2's correction) unless the problem asks for a specific one (lexicographically smallest -> min-heap instead of plain queue).

Shortest-path recognition:

GRAPH problem

need reachability / groups?

explore nodes

-> DFS / BFS

repeated merge/query

-> DSU

need shortest path?

unweighted

-> BFS

0/1 weights

-> 0-1 BFS

non-negative

-> Dijkstra

negative edges

-> Bellman-Ford

need ALL-pairs shortest paths?

-> Floyd-Warshall

(see the MST / dependency / hidden-graph tree for the rest)

MST / dependency / hidden-graph recognition:

connect ALL nodes, minimum total cost?

edge list -> Kruskal + DSU grow from a vertex -> Prim + heap

dependencies / prerequisites?

-> Topological Sort (Kahn's or DFS postorder)

use every EDGE exactly once?

-> Eulerian path / Hierholzer's

problem doesn't look like a graph at all?

-> find the hidden STATE + TRANSITION, then BFS/DFS/Dijkstra

extra constraint? expand the state: (node, constraint)

DSU recognition checklist -- ask these in order:

1. Are entities being connected/merged? -> DSU candidate.
2. Do I repeatedly need "same group?" queries? -> strong signal.
3. Are edges processed one at a time (not all given upfront)? -> strong signal.
4. Does an edge connecting already-connected nodes mean a cycle/redundancy? -> DSU is likely the tool.

And when **not** to reach for DSU: need the actual path, need shortest distance, need traversal order or levels, need dependency ordering, need detailed component traversal (not just membership), or general directed reachability -- none of these are things DSU tracks.

14.2 Common mistakes (collected from this guide's own corrections)

- Assuming topological sort / Kahn's algorithm produces a **unique** order -- it usually doesn't (Part 4.2).
- Assuming a visited neighbor always means a cycle in an **undirected** graph -- only true if it isn't the parent (Part 2.2).
- Using Dijkstra when negative edges are possible -- its finalization step is unsound the moment a negative edge exists (Part 7.5).
- Updating Bellman-Ford-with-K-stops' distance array **in place** within a single round, letting one round silently chain more than the intended number of extra edges (Part 10.1).
- Confusing Dijkstra's heap priority (total path distance) with Prim's (just the one connecting edge) -- their code looks nearly identical (Part 8.5).
- Forgetting DSU only tracks connectivity, never distance or path (Part 5.7).

14.3 Python templates cheat sheet

```

# Adjacency list (unweighted, undirected)
graph = [[] for _ in range(n)]
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)

# DSU skeleton
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True

# Kahn's topological sort
indegree = [0] * n
queue = deque(i for i in range(n) if indegree[i] == 0)

# Dijkstra
heap = [(0, start)]
while heap:
    d, node = heapq.heappop(heap)
    if d > dist[node]:
        continue
    ...

# 0-1 BFS
dq.appendleft(nei) if weight == 0 else dq.append(nei)

```

14.4 Time / space complexity cheat sheet

Technique	Time	Space
DFS / BFS traversal	$O(V+E)$	$O(V)$
DSU (path compression + union by size/rank)	$O(E * \alpha(V)) \sim O(E)$	$O(V)$
Kahn's / DFS topological sort	$O(V+E)$	$O(V+E)$
Dijkstra (binary heap)	$O(E \log V)$	$O(V)$

Technique	Time	Space
0-1 BFS	$O(V+E)$	$O(V)$
Bellman-Ford	$O(V \cdot E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal	$O(E \log E)$ (sort)	$O(V)$
Prim (binary heap)	$O(E \log V)$	$O(V)$
Bellman-Ford with K stops	$O(K \cdot E)$	$O(V)$

Part 15: Interview Q&A -- Mixed Recognition Drills

The source chat was explicit about why this matters:

Occasionally give me a problem without telling me whether it uses DFS, BFS, DSU, Dijkstra, topological sort, MST, 0-1 BFS, or something else. Let me decide. I don't want to develop the habit "I'm studying DSU, therefore the answer must be DSU." I want to recognize the underlying structure independently.

Reason through each one *before* reading its answer.

Q1. n computers, some already directly connected. Connections arrive one at a time as (a, b) pairs, and after each one you must report how many separate networks remain. What technique, and why?

A1. DSU. The signal is "connections arrive one at a time" plus "repeated group-count queries" -- exactly the dynamic-connectivity pattern from Part 5.1, not a graph you get to explore once.

Q2. Given a directed graph of n tasks and dependency pairs, find **one** valid execution order, or report that none exists.

A2. Topological sort (Kahn's or DFS postorder) -- "dependency" + "order" is the strongest possible signal from Part 4. Either technique works; if lexicographically-smallest order is required, use a min-heap instead of a plain queue.

Q3. A weighted, undirected graph where all weights are positive. Find the minimum total edge weight required to connect every vertex.

A3. MST -- Kruskal or Prim, from Part 8. The giveaway phrase is "connect every vertex" + "minimum total," not "from A to B."

Q4. Same graph as Q3, but now: find the cheapest route from vertex A to vertex D specifically.

A4. Dijkstra (all weights are positive/non-negative) -- Part 7.3. Note this is the *same* input graph as Q3 but a *different* question, and the two require genuinely different algorithms -- this pair is designed to catch the "I'm studying MST so the answer must be MST" trap directly.

Q5. A word ladder-style problem, except some transformations are free (cost 0, e.g. changing case) and others cost 1 (changing the letter). Find the cheapest transformation sequence.

A5. 0-1 BFS (Part 7.4) -- weights restricted to $\{0, 1\}$ is the exact trigger, and it's strictly faster than reaching for Dijkstra here even though Dijkstra would also give the right answer.

Q6. Determine whether a directed graph can be split so that every node either reaches a designated "safe" terminal set, or is itself terminal, with no node stuck in an infinite loop.

A6. Three-color DFS, the Find Eventual Safe States pattern from Part 13.4 -- "no node stuck in an infinite loop" is a cycle-reachability question, not a shortest-path or connectivity one.

Q7. n cities, and a proposed list of new roads with their costs. Determine the minimum cost to guarantee every city can reach every other city, given some cities may already be connected via existing roads.

A7. MST via Kruskal + DSU (Part 8.7's water-distribution pattern generalizes here too) -- pre-existing connections just mean some DSU unions happen "for free" before considering the priced candidate edges.

Q8. A grid of rooms, some locked, where entering a room may reveal keys to previously locked rooms. Determine if you can reach a specific target room.

A8. Expanded-state BFS/DFS (Part 10.2) -- the state isn't just "which cell am I in," it's "which cell am I in, given which keys I've collected so far." Same underlying move as the coupon and alternating-color problems, just with a different constraint.

Q9. Given flight routes with prices, and a limit on the number of layovers allowed, find the cheapest itinerary.

A9. Cheapest Flights Within K Stops -- Bellman-Ford-style layered relaxation (Part 10.1), *not* plain Dijkstra, because the stop-count limit has to become part of what's being optimized over.

Q10. A set of email accounts where some share an email address across different listed names. Group them by actual owner.

A10. DSU, the Accounts Merge pattern (Part 6.2) -- "shared identifier -> same group" is the tell, even though the problem never mentions graphs or components explicitly.

Closing recap

If there's one instinct to leave this guide with, it's the source chat's own recognition ladder, repeated once more because it's the thing that actually transfers to problems you haven't seen before:

"Can I reach?" -> DFS/BFS. "Are these already connected?" -> DSU. "Does this edge create a cycle?" -> DSU. "Shortest, unweighted?" -> BFS. "Shortest, 0/1?" -> 0-1 BFS. "Shortest, non-negative?" -> Dijkstra. "Negative edges?" -> Bellman-Ford. "All pairs?" -> Floyd-Warshall. "Cheapest way to connect EVERYTHING?" -> MST. "Prerequisites?" -> Topological Sort. "Use every EDGE once?" -> Eulerian/Hierholzer. "Doesn't look like a graph?" -> find the hidden STATE. "Shortest path + a constraint?" -> expand the state.